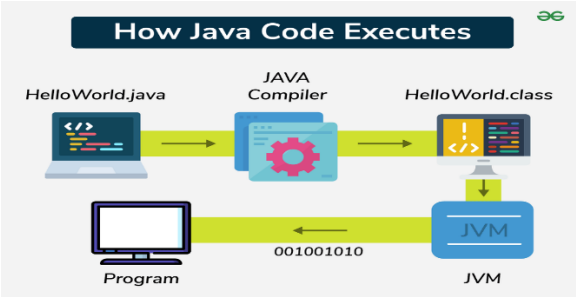
INTRODUCTION TO JAVA:

Java is a high-level, object-oriented programming language designed by Sun Microsystems (now Oracle) in the mid-1990s. Its key strengths include **platform independence** and **"write once, run anywhere" (WORA)** capability. Java programs are first compiled into bytecode, which can run on any machine with a Java Virtual Machine (JVM).



Java source (`.java`) is compiled by `javac` into bytecode (`.class`), and then the JVM loads and executes the bytecode.

CREATION AND EVOLUTION OF JAVA

The Java programming language was created in the early 1990s by a team at Sun Microsystems led by James Gosling. The project, initially codenamed "Green Project" and the language "Oak," was originally intended for interactive television and other digital consumer devices.

The primary goal was to create a language that was platform-independent, meaning it could run on various types of hardware without needing to be recompiled for each one.

The team considered using C++ but rejected it due to its complexity and memory requirements. Gosling's objective was to design a language with a C/C++-like syntax but with greater simplicity and uniformity.

The project's focus shifted towards the growing World Wide Web, and in 1995, the language was renamed "Java" and officially released to the public as Java 1.0. Its core promise was **"Write Once, Run Anywhere" (WORA)**, which resonated deeply with developers building applications for the internet.

Java's evolution has been marked by significant updates that expanded its capabilities:

J2SE 1.2 (1998): Introduced the Swing GUI toolkit and the Collections Framework.

J2SE 5.0 (2004): A major release that added generics, annotations, autoboxing, and the enhanced for-each loop, making the language more powerful and expressive.

Java 8 (2014): A landmark release that introduced lambda expressions and the Stream API, bringing functional programming paradigms to Java and dramatically changing how developers process collections of data.

Modern Release Cycle (2018 onwards): Oracle adopted a faster, six-month release cycle, with certain versions designated as Long-Term Support (LTS) releases, such as Java 11, 17, and 21.

Today, Java is a dominant in enterprise-level applications, web backends, big data systems, and is the primary language for Android mobile app development.

CORE FEATURES OF THE JAVA LANGUAGE

Object-Oriented: Java is a pure object-oriented language. Everything in Java is an object (with the exception of primitive data types), and programs are built by creating and interacting with these objects. It supports all fundamental OOP principles like encapsulation, inheritance, and polymorphism.

Platform Independent: This is arguably Java's most famous feature. Java code is compiled into an intermediate format called **bytecode**, not into machine-specific code. This bytecode can run on any machine that has a Java Virtual Machine (JVM), fulfilling the "Write Once, Run Anywhere" promise.

Simple and Easy to Learn: Java was designed to be relatively easy to learn for programmers already familiar with C and C++. It omits complex and error-prone features like explicit pointers and manual memory management.

Secure: The Java platform provides several security features. Code runs inside the JVM, which acts as a sandbox, preventing it from accessing the underlying operating system directly. The absence of explicit pointers prevents unauthorized memory access, and a bytecode verifier checks the code for illegal operations before it is run.

Robust: Java is a robust language, emphasizing both compile-time error checking and runtime exception handling. Its automatic memory management, handled by the Garbage Collector, prevents common programming errors like memory leaks.

Multithreaded: Java has built-in support for multithreading, which allows a program to perform multiple tasks concurrently. This is crucial for building responsive and high-performance applications.

High Performance: While interpreted languages are often slower than compiled ones, Java achieves high performance through its Just-In-Time (JIT) compiler. The JIT compiler converts frequently executed bytecode into native machine code at runtime, significantly speeding up execution.

COMPARISON WITH C++

Feature	Java	C++
Platform	Platform-independent (runs on JVM – write once, run anywhere)	Platform-dependent (compiled directly to machine code, needs recompilation for each OS)
Memory Management	Automatic Garbage Collection	Manual (programmer must use new and delete)
Programming Paradigm	Purely Object-Oriented (almost everything is inside a class, except primitives)	Multi-paradigm: supports both Object-Oriented and Procedural programming
Pointers	Does not support explicit pointers (for security)	Supports pointers and direct memory manipulation
Multiple Inheritance	Not supported directly (achieved via interfaces)	Supported (a class can inherit from multiple classes)
Operator Overloading	Not supported	Supported

Feature	Java	C++
Compilation & Execution	Compiled into bytecode, then executed by JVM	Compiled directly into machine code by the compiler
Speed	Slower (because of JVM overhead)	Faster (directly compiled to machine code)
Libraries	Rich built-in libraries (Java API)	Standard Template Library (STL)
Exception Handling	Mandatory for checked exceptions	Not mandatory
Platform Dependence	Runs on any machine with JVM	Runs only on the platform it was compiled for

THE JAVA VIRTUAL MACHINE (JVM) AND BYTECODE

The Java Virtual Machine (JVM) enables Java's platform independence by providing an environment to execute Java bytecode.

Bytecode: When a Java source file (.java) is compiled, the javac compiler translates it into an intermediate language called bytecode, which is stored in a .class file. This bytecode is a set of instructions designed for the JVM, not for a specific physical processor.

JVM ARCHITECTURE

The JVM has a well-defined internal architecture consisting of three main components: the Class Loader Subsystem, the Runtime Data Areas, and the Execution Engine.

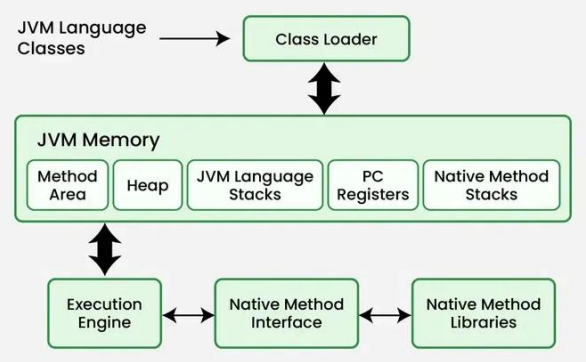


Figure: Architecture of the Java Virtual Machine (JVM)

Class Loader Subsystem: This component is responsible for dynamically loading Java classes into the JVM at runtime. This process involves three phases:

1. **Loading:** The Class Loader finds and imports the binary data for a class (the .class file). This is done by a hierarchy of loaders: the Bootstrap Class Loader (for core Java libraries), the Extension Class Loader, and the Application Class Loader (for your application's classes).
2. **Linking:** This phase combines the loaded class into the runtime state of the JVM. It includes:
    a. **Verification:** The bytecode verifier ensures the code is valid and does not violate Java's security constraints.
    b. **Preparation:** Memory is allocated for static variables, and they are initialized with their default values.

   c. **Resolution:** Symbolic references within the code (like class names) are replaced with direct memory references.
3. **Initialization:** The final phase, where the static initializers (static blocks) of the class are executed, and static variables are assigned their actual starting values.

Runtime Data Areas: These are the memory areas the JVM uses during program execution.

- **Method Area:** A shared memory area that stores class-level information, such as class metadata, the runtime constant pool, and the code for methods and constructors. Static variables are also stored here.
- **Heap:** Another shared memory area where all objects and arrays are dynamically allocated at runtime. This is the area managed by the Garbage Collector.
- **Stack Area:** Each thread of execution has its own private JVM Stack. When a method is invoked, a new "stack frame" is created and pushed onto the stack for that thread. The frame holds local variables, operand stacks (for intermediate calculations), and frame data. The stack is destroyed when the thread terminates.
- **PC (Program Counter) Registers:** Each thread has its own PC Register. It contains the address of the JVM instruction currently being executed. If the method is native, the PC register is undefined.
- **Native Method Stack:** This stack supports native methods (methods written in languages other than Java, such as C/C++). It is separate from the Java Stack and is specific to each thread.

Execution Engine: This component is responsible for executing the bytecode loaded into the runtime data areas. It contains:

- **Interpreter:** Reads, interprets, and executes bytecode instructions one by one. It is quick to start but can be slow for repetitive code.
- **Just-In-Time (JIT) Compiler:** To improve performance, the JVM's execution engine includes a JIT compiler. It identifies "hot spots"—parts of the bytecode that are executed frequently—and compiles them into native machine code at runtime. This native code is then cached and executed directly, resulting in significant performance gains. This synergy allows Java to start quickly via interpretation and achieve high performance for long-running applications via JIT compilation.
- **Garbage Collector (GC):** The GC is a background process that automatically manages memory in the heap. It tracks which objects are still in use and reclaims the memory occupied by unreachable objects, preventing memory leaks.

BASIC PROGRAM STRUCTURE

Every executable Java program must contain a class and a main method. The main method serves as the entry point for the application.

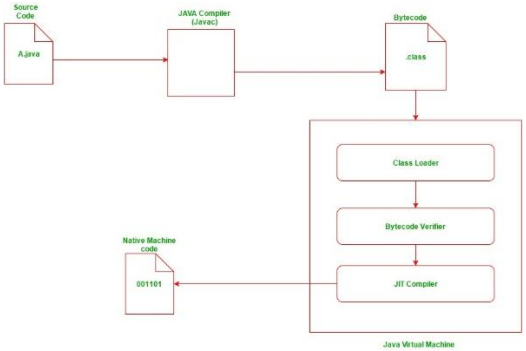
```
// This is a simple Java program.
// File name: HelloWorld.java

public class HelloWorld {
   // The main method is where program execution begins.
   public static void main(String args) {
      // This statement prints "Hello, World!" to the console.
      System.out.println("Hello, World!");
   }
}
```

This simple program demonstrates the basic structure:

- A public class definition that matches the filename (HelloWorld.java).
- The main method with its specific signature.
- A statement inside the main method that performs an action.

COMPILATION AND EXECUTION PROCESS

Java employs a two-step process to run a program, which is the key to its platform independence.



Compilation: The source code, written in a .java file, is passed to the Java compiler (javac). The compiler checks the code for syntax errors and, if successful, translates it into platform-independent bytecode, saving it as a .class file.

Command: javac HelloWorld.java

Execution: The java command is used to launch the JVM. The JVM loads the specified .class file, verifies the bytecode for security, and then executes it using its Execution Engine (Interpreter and JIT compiler). The JVM translates the bytecode into native machine instructions that the underlying operating system and hardware can understand and execute.

Command: java HelloWorld

THE MAIN() METHOD EXPLAINED

The main method has a very specific signature that the JVM is designed to recognize as the starting point of an application. Let's break down each part of public static void main(String args).

- public: This is an access modifier. The main method must be public so that it can be called by the JVM, which exists outside of the program's package.
- static: This keyword means the method belongs to the class itself, not to a specific instance (object) of the class. The JVM calls the main method before any objects are created, so it must be static to be accessible.
- void: This is the return type. void signifies that the main method does not return any value after it completes execution. When main finishes, the program terminates.
- main: This is the name of the method. The JVM is hardwired to look for a method with this exact name as the entry point.
- String args: This is the single parameter of the method. It is an array of String objects that allows you to pass command-line arguments to your program when you run it. Each argument is passed as a separate string in the args array.

LANGUAGE LEXICON (LEXICAL ISSUES)

A Java program is composed of a sequence of tokens. Understanding these fundamental building blocks, or lexical issues, is essential for writing valid code.[17]

- **Whitespace:** Java is a free-form language, meaning that spaces, tabs, and newlines are generally ignored by the compiler. However, they are crucial for separating tokens and formatting code for human readability.

- **Identifiers:** These are the names given to classes, methods, variables, and packages. An identifier must start with a letter, an underscore (_), or a dollar sign ($). Subsequent characters can be letters, digits, underscores, or dollar signs. Identifiers are case-sensitive (myVariable is different from MyVariable). By convention, variable and method names use camelCase, while class names use PascalCase.
- **Keywords:** These are reserved words that have a predefined meaning in the Java language and cannot be used as identifiers. Examples include class, public, static, int, if, for, and while.
- **Literals:** A literal is a source code representation of a fixed value.
    - Integer literals: 10, -200
    - Floating-point literals: 3.14, 2.5f
    - Character literals: 'A', '\n' (enclosed in single quotes)
    - String literals: "Hello, Java!" (enclosed in double quotes)
    - Boolean literals: true, false
    - Null literal: null [17]
- **Separators:** These are symbols that define the structure and grouping of code.
    - (): Used for method calls and parameter lists.
    - {}: Defines code blocks and class/method bodies.
    - `` : Used for declaring and accessing arrays.
    - **;** : Terminates statements.
    - **,** : Separates identifiers in a variable declaration or items in a method's argument list.
    - **.** : Used to access members of a class or package.[29]
- **Comments:** Comments are explanatory text ignored by the compiler.
    - // Single-line comment: Comments out text until the end of the line.
    - /* Multi-line comment */: Comments out a block of text.
    - /** Javadoc comment */: A special type of comment used to generate API documentation.

SETTING UP YOUR ENVIRONMENT

To compile and run Java programs, you need the Java Development Kit (JDK).

INSTALLING THE JDK

- **Uninstall Old Versions:** It is recommended to uninstall any older versions of the JDK or Java Runtime Environment (JRE) to avoid conflicts.
- **Download the JDK:** Visit the official Oracle Java download site and download the installer for your operating system (e.g., the "x64 Installer" for Windows).
- **Run the Installer:** Execute the downloaded installer and follow the on-screen instructions. It is usually best to accept the default settings. The default installation path on Windows is typically C:\Program Files\Java\jdk-version.

CONFIGURING THE PATH VARIABLE

The PATH is an environment variable that your operating system uses to locate executable files. Setting the PATH allows you to run Java commands like javac and java from any directory in your command prompt or terminal without typing the full path to the executable.

Steps for Windows:

1. **Find the bin directory:** Navigate to your JDK installation directory (e.g., C:\Program Files\Java\jdk-21) and open the bin folder. Copy this full path to your clipboard.
2. **Open Environment Variables:** In the Windows search bar, type "environment variables" and select "Edit the system environment variables." In the System Properties window, click the "Environment Variables..." button.
3. **Edit the Path variable:** In the "System variables" section, find the variable named Path, select it, and click "Edit...".
4. **Add the JDK path:** Click "New" and paste the path to the JDK's bin directory that you copied earlier. Click OK to save all changes.

5. **Verify the installation:** Open a new command prompt and type java -version and javac -version. If the installation was successful, you will see the version information for the Java runtime and compiler.

It is also common practice to set a JAVA_HOME environment variable that points to the root JDK installation directory (e.g., C:\Program Files\Java\jdk-21). Many development tools like Maven, Gradle, and IDEs use JAVA_HOME to locate the JDK.

CORE BUILDING BLOCKS: DATA TYPES, VARIABLES, AND ARRAYS

PRIMITIVE DATA TYPES

Java is a strongly typed language, meaning every variable and expression has a type that is known at compile time. This helps detect errors early. Java has two categories of data types: primitive and reference.

Primitive types are the most basic data types available. There are eight of them.

Type	Size	Description	Default Value
byte	8-bit	Signed two's complement integer. Range: -128 to 127.	0
short	16-bit	Signed two's complement integer. Range: -32,768 to 32,767.	0
int	32-bit	Signed two's complement integer. Range: -2,147,483,648 to 2,147,483,647.	0
long	64-bit	Signed two's complement integer. Used for very large numbers. Must be suffixed with L.	0L
float	32-bit	Single-precision IEEE 754 floating-point number. Must be suffixed with f.	0.0f
double	64-bit	Double-precision IEEE 754 floating-point number. The default for decimal values.	0.0d
char	16-bit	A single Unicode character. Range: \u0000 to \uffff.	\u0000
boolean	1-bit (logically)	Represents two values: true or false.	false

VARIABLES

A variable provides a named storage location in memory.

Declaration and Initialization: A variable must be declared with a type before it can be used. It can be initialized at the time of declaration or later.

```
int score; // Declaration
score = 100; // Initialization
double pi = 3.14159; // Declaration and initialization in one step
```

- **Scope and Lifetime:** The scope of a variable determines where it can be accessed, and its lifetime is the period during which it exists in memory.

- **Local Variables:** Declared within a method, constructor, or code block. Their scope is limited to that block. They must be initialized before use as they have no default value.
- **Instance Variables:** Declared within a class but outside any method. Each object (instance) of the class has its own copy of these variables. They have default values and exist for the lifetime of the object.
- **Static (Class) Variables:** Declared with the static keyword. Only one copy of a static variable exists, shared among all instances of the class. It exists for the lifetime of the program.

ARRAYS

In Java, an array is an important linear data structure that allows us to store multiple values of the same type.

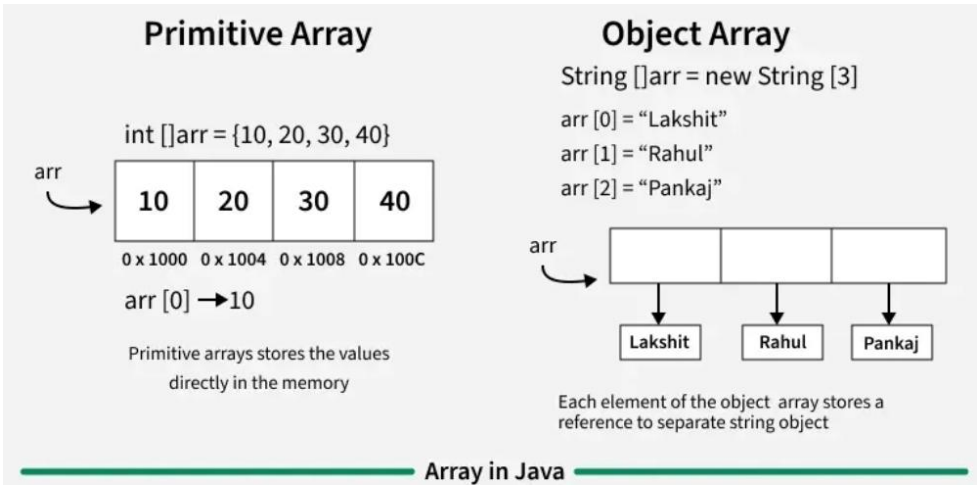
Arrays in Java are objects, like all other objects in Java, arrays implicitly inherit from the java.lang.Object class. This allows you to invoke methods defined in Object (such as toString(), equals() and hashCode()).

Arrays have a built-in length property, which provides the number of elements in the array.

```
public class Mca {
   public static void main(String[] args) {
      // initializing array
      int[] arr = { 40,55,63,17,22,68,89,97,89};

      // size of array
      int n = arr.length;

      // traversing array
      for (int i = 0; i < n; i++)
         System.out.print(arr[i] + " ");
   }
}
```

FEATURES OF ARRAYS:

- **Store Data**: Arrays can hold both primitives (int, char, boolean) and objects (String, Integer, etc.).
- **Memory**: Primitives are stored in continuous memory; objects store references continuously.
- **Indexing**: Starts at 0.
- **Fixed Size**: Size is set when created and cannot change.



BASICS OPERATION ON ARRAYS IN JAVA

1. DECLARING AN ARRAY

The general form of array declaration is

```
// Method 1:
int arr[ ];

// Method 2:
int[ ] arr;
```

The element type determines the data type of each element that comprises the array. Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc. or user-defined data types (objects of a class).

Note: It is just how we can create is an array variable, no actual array exists. It merely tells the compiler that this variable (int Array) will hold an array of the integer type.

2. INITIALIZATION AN ARRAY IN JAVA

When an array is declared, only a reference of an array is created. We use new to allocate an array of given size.

```
int arr[ ] = new int[size];
```

Array Declaration is generally static, but if the size in not defined, the Array is Dynamically sized.

Memory for arrays is always dynamically allocated (on heap segment) in Java. This is different from C/C++ where memory can either be statically allocated or dynamically allocated.

The elements in the array allocated by new will automatically be initialized to zero (for numeric types), false (for boolean) or null (for reference types).

ARRAY LITERAL IN JAVA

In a situation where the size of the array and variables of the array are already known, array literals can be used.

```
// Declaring array literal
int[ ] arr = new int[ ]{ 1,2,3,4,5,6,7,8,9,10 };
```

The length of this array determines the length of the created array.

There is no need to write the new int[] part in the latest versions of Java.

3. CHANGE AN ARRAY ELEMENT

To change an element, assign a new value to a specific index. The index begins with 0 and ends at (total array size)-1.

```
// Changing the first element to 90
arr[0] = 90;
```

4. ARRAY LENGTH

We can get the length of an array using the length property:

```
// Getting the length of the array
int n = arr.length;
```

MULTIDIMENSIONAL ARRAYS

In Java, multidimensional arrays are arrays of arrays. The most common is a two-dimensional (2D) array which can be thought of as a table (rows × columns).

DECLARATION

```
int[ ][ ] matrix;      // declaration
matrix = new int[3][4]; // 3 rows, 4 columns
```

INITIALIZATION

```
int[ ][ ] mat = {
   {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9}
};
```

ACCESSING ELEMENTS

System.out.println(mat[0][2]); // prints 3

TRAVERSING A 2D ARRAY

```
for (int i = 0; i < mat.length; i++) {
   for (int j = 0; j < mat[i].length; j++) {
      System.out.print(mat[i][j] + " ");
   }
   System.out.println();
}
```

JAGGED ARRAYS

Rows can have different lengths.

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];   // row 1 has 2 cols
jagged[1] = new int[3];   // row 2 has 3 cols
jagged[2] = new int[1];   // row 3 has 1 col
```

ADVANTAGES OF ARRAYS

- Easy to access elements using index.
- Faster because of contiguous memory allocation.
- Better performance for iteration compared to lists.

LIMITATIONS OF ARRAYS

- Fixed size (cannot grow/shrink dynamically).
- Insertion and deletion are costly (require shifting).
- Cannot directly store elements of different data types.

SHORT PROGRAMS (IMPORTANT FOR EXAMS)

Program to find sum of array elements:

```
class SumArray {
   public static void main(String[] args) {
      int[] arr = {10, 20, 30, 40};
      int sum = 0;
      for (int i = 0; i < arr.length; i++) {
         sum += arr[i];
      }
      System.out.println("Sum = " + sum);
   }
}
```

Program to print 2D array:

```
class TwoDArray {
   public static void main(String[] args) {
      int[][] mat = {{1,2}, {3,4}, {5,6}};
      for(int i=0; i<mat.length; i++) {
         for(int j=0; j<mat[i].length; j++) {
            System.out.print(mat[i][j] + " ");
         }
         System.out.println();
      }
   }
}
```

TYPE CONVERSION AND EXPRESSION PROMOTION

Type conversion means changing a value from one data type to another.

There are two types:

1.WIDENING OR AUTOMATIC TYPE CONVERSION

Widening conversion takes place when two data types are automatically converted. This happens when:

- The two data types are compatible.
- When we assign a value of a smaller data type to a bigger data type.

Happens automatically by the compiler.

Smaller data type → bigger data type.

No data loss.

For Example, in java, the numeric data types are compatible with each other but no automatic conversion is supported from numeric type to char or boolean. Also, char and boolean are not compatible with each other.

Ex:

```
int a = 10;
double b = a; // int automatically converted to double
```

Order of widening:

Byte –> Short –> Int –> Long – > Float –> Double

Widening or Automatic Conversion

2.NARROWING OR EXPLICIT CONVERSION

If we want to assign a value of a larger data type to a smaller data type we perform explicit type casting or narrowing.

- This is useful for incompatible data types where automatic conversion cannot be done.
- Here, the target type specifies the desired type to convert the specified value to.

Double –> Float –> Long –> Int –> Short –> Byte

Narrowing or Explicit Conversion

char and number are not compatible with each other. Let's see when we try to convert one into another.

```
// Explicit Type Conversion
// Main class
public class GFG {
   // Main driver method
   public static void main(String[] argv)
   {
      // Declaring character variable
      char ch = 'c';
      // Declaring integer variable
      int num = 88;
      // Trying to insert integer to character
      ch = num;
   }
}
```

Output: An error will be generated

Ex:

```
double x = 9.78;
int y = (int) x; // 9  double converted to int, decimal part lost
```

TYPE PROMOTION IN EXPRESSIONS

When Java evaluates an expression, the **operands may be promoted to a common data type** to avoid data loss.

While evaluating expressions, the intermediate value may exceed the range of operands and hence the expression value will be promoted. Some conditions for type promotion are:

- Java automatically promotes each byte, short, or char operand to int when evaluating an expression.
- If one operand is long, float or double the whole expression is promoted to long, float, or double respectively.

Rules:

byte, short, char → int (promoted automatically when used in expressions).

```
byte a = 10, b = 20;
byte c = (byte)(a + b); // a+b promoted to int
```

If one operand is long, result becomes long.

```
int x = 10;
long y = 20;
```

```
long res = x + y; // result is long
```

If one operand is float, result becomes float.

If one operand is double, result becomes double.

This is called **type promotion in expressions.**

OPERATORS, EXPRESSIONS, AND CONTROL STATEMENTS

OPERATORS

Operators are special symbols that perform operations on variables and values.

- **Arithmetic Operators:** + (addition), - (subtraction), * (multiplication), / (division), % (modulus), ++ (increment), -- (decrement).
- **Relational Operators:** Used for comparison. They return a boolean value. == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), <= (less than or equal to).
- **Logical Operators:** Used to combine boolean expressions. && (logical AND), || (logical OR), ! (logical NOT). The && and || operators are "short-circuiting," meaning they do not evaluate the right-hand operand if the result can be determined from the left-hand one.
- **Bitwise Operators:** Perform operations on the individual bits of integer types. & (bitwise AND), | (bitwise OR), ^ (bitwise XOR), ~ (bitwise NOT), << (left shift), >> (signed right shift), >>> (unsigned right shift).
- **Assignment Operators:** Assign values to variables. = (simple assignment), +=, -=, *=, /=, %= (compound assignment).
- **Ternary Operator:** A shorthand for an if-else statement. (condition)? value_if_true : value_if_false.

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Operator precedence determines the order in which operators in an expression are evaluated. Associativity determines the order for operators with the same precedence.

Precedence	Operator Category	Operators	Associativity
1 (Highest)	Postfix	expr++, expr--	Left to Right
2	Unary	++expr, --expr, +, -, ~, !	Right to Left
3	Multiplicative	*, /, %	Left to Right
4	Additive	+, -	Left to Right
5	Shift	<<, >>, >>>	Left to Right
6	Relational	>, >=, <, <=, instanceof	Left to Right
7	Equality	==, !=	Left to Right
8	Bitwise AND	&	Left to Right
9	Bitwise XOR	^	Left to Right

10	Bitwise OR	`	`
11	Logical AND	&&	Left to Right
12	Logical OR	`	
13	Ternary	? :	Right to Left
14 (Lowest)	Assignment	=, +=, -=, *=, /= etc.	Right to Left

CONTROL STATEMENTS

A programming language uses control statements to control the flow of execution of a program based on certain conditions.. Java provides several control statements to manage program flow, including:

- Conditional Statements: if, if-else, nested-if, if-else-if
- Switch-Case: For multiple fixed-value checks
- Jump Statements: break, continue, return

SELECTION STATEMENTS:

If Statement:

The if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e. if a certain condition is true then a block of statements is executed otherwise not.

SYNTAX:

```
if(condition) {
// Statements to execute if
// condition is true
}
```

If we don't use curly braces({}), only the next line after the if is considered as part of the if block For example,

```
if (condition) // Assume condition is true
statement1; // Belongs to the if block
statement2; // Does NOT belong to the if block
```

Here's what happens:

- If the condition is True statement1 executes.
- statement2 runs no matter what because it's not a part of the if block

Ex:

```
import java.util.*;
class Mca {
   public static void main(String args[])
   {
      int i = 10;
      if (i < 15)
         // part of if block(immediate one statement after if condition)
         System.out.println("Inside If block");

      // always executes as it is outside of if block
      System.out.println("10 is less than 15");
```

```
    }
}
```

if-else statement:

Executes a block of code if a condition is true, and an optional else block if it is false.

Syntax:

```
if(condition){
// Executes this block if
// condition is true
}else{
// Executes this block if
// condition is false
}
```

Example:

```
class Mca {
   public static void main(String args[])
   {
      int i = 10;
      if (i < 15)
         System.out.println("i is smaller than 15");
      else
         System.out.println("i is greater than 15");
 }}
```

JAVA NESTED-IF STATEMENT

Nested if statements mean an if statement inside an if statement. Yes, java allows us to nest if statements within if statements.

Syntax:

```
if (condition1) {
// Executes when condition1 is true
if (condition2){
 // Executes when condition2 is true
}
}
```

Example:

```
class Mca {
   public static void main(String args[]) {
      int i = 10;
      // Outer if statement
      if (i < 15) {
         System.out.println("i is smaller than 15");
         // Nested if statement
         if (i == 10) {
            System.out.println("i is exactly 10");
         }
      }
   }
}
```

Output

```
i is smaller than 15

i is smaller than 12 too
```

IF-ELSE-IF LADDER:

- Used when multiple conditions need to be checked.
- Conditions are tested top to bottom.
- When one condition is true, its block runs and the rest are skipped.
- If no condition is true → else block runs.
- Only one else is allowed, but you can have many else if.

Syntax:

```
if (condition1) {
  // runs if condition1 true
} else if (condition2) {
  // runs if condition2 true
} else {
  // runs if all false
}
```

SWITCH:

A multi-way branch statement that executes code based on the value of an expression. It is often more efficient than a long if-else-if ladder for checking a single variable against multiple constant values.

Syntax:

```
switch (expression) {
    case value1:
          // code to be executed if expression == value1
          break;
    case value2:
           // code to be executed if expression == value2
            break;
   // more cases...
   default:
          // code to be executed if no cases match
}
```

Example:

```
int day = 4;
switch (day) {
  case 6:
    System.out.println("Saturday");
    break;
  case 7:
    System.out.println("Sunday");
    break;
  default:
    System.out.println("Weekday");
}
```

- The expression can be of type byte, short, int char, or an enumeration. Beginning with JDK7, the expression can also be of type String.
- Duplicate case values are not allowed.
- The default statement is optional.
- The break statement is used inside the switch to terminate a statement sequence.
- The break statements are necessary without the break keyword, statements in switch blocks fall through.
- If the break keyword is omitted, execution will continue to the next case.

ITERATION STATEMENTS (LOOPS):

Loops in programming allow a set of instructions to run multiple times based on a condition. In Java, there are three types of Loops, which are explained below:

1. FOR LOOP

The for loop is used when we know the number of iterations (we know how many times we want to repeat a task). The for statement includes the initialization, condition, and increment/decrement in one line.

Syntax:

```
for (initialization; condition; increment/decrement) {
// code to be executed
}
// Java program to demonstrates the working of for loop
import java.io.*;
class Mca {
   public static void main(String[] args)
   {
      for (int i = 0; i <= 10; i++) {
         System.out.print(i + " ");
      }
   }
}
```

- **Initialization**: Declare the loop variable (done once at start).
- **Condition**: Checked before each iteration; must be true to continue.
- **Execution**: Loop body runs if condition is true.
- **Update**: Variable is incremented/decremented after each iteration.
- **Termination**: Loop ends when condition becomes false.

2. WHILE LOOP

A while loop is used when we want to check the condition before executing the loop body. We usually use this loop when we don't know the number of iterations in advance.

Syntax:

```
while (condition) {
// code to be executed
}
```

Example:

```
import java.io.*;
class Mca {
   public static void main(String[] args) {
      int i = 0;
```

```
    while (i <= 10) {
        System.out.print(i + " ");
        i++;
    }}}
```

- **Condition Check**: While loop starts by checking a boolean condition.
- **Execution**: If true, loop body runs; if false, control moves outside the loop.
- **Update**: Usually the body updates the variable for the next check.
- **Termination**: Loop ends when the condition becomes false.

3. DO-WHILE LOOP

The do-while loop ensures that the code block executes at least once before checking the condition.

- **First Execution**: Statements run once without checking any condition.
- **Condition Check**: After execution and update, condition is tested.
- **Repeat**: If true, loop continues; if false, it ends.
- **Key Point**: Always runs at least once → it's an **exit control loop**.

Syntax:

```
do {
// code to be executed
} while (condition);
```

Example:

```
import java.io.*;
class Mca {
   public static void main(String[] args)
   {
      int i = 0;
      do {
         System.out.print(i + " ");
         i++;
      } while (i <= 10);
   }
}
```

FOR-EACH LOOP

There is another form of the for loop known as Enhanced for loop or (for each loop).This loop is used to iterate over arrays or collections.

Syntax:

```
for (dataType variable : arrayOrCollection) {
// code to be executed
}
```

Example:

```
import java.io.*;
class Mca {
   public static void main(String[] args)
   {
```

```
        String[] names = { "Sweta", "Gudly", "Amiya" };

        for (String name : names) {
            System.out.println("Name: " + name);
        }
    }
}
```

JUMP STATEMENTS

Java supports three jump statements: break, continue and return. These three statements transfer control to another part of the program.

Break: In Java, a break is majorly used for:

- Terminate a sequence in a switch statement (discussed above).
- To exit a loop.
- Used as a "civilized" form of goto.

Continue:

The continue statement in Java is used inside loops (for, while, do-while). It skips the current iteration of the loop and moves control to the next iteration.

How it works:

- In a for loop → continue jumps to the update statement.
- In a while / do-while loop → continue jumps back to the condition check.

Syntax:

```
continue;
```

Example:

```
import java.util.*;
class Mca {
    public static void main(String args[])
    {
        for (int i = 0; i < 10; i++) {
            // If the number is even, skip and continue
            if (i % 2 == 0)
                continue;
            // If number is odd, print it
            System.out.print(i + " ");
        }
    }
}
```

RETURN STATEMENT

The return statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method.

Syntax:

```
return;          // for void methods
return value;    // for methods with return type
```

Examples:

(a) Returning a value

```
int sum(int a, int b) {
```

```
    return a + b; // returns int value
}
```

(b) Void method with return

```
void display() {
    System.out.println("Hello!");
    return; // not necessary, but can be used to exit early
}
```

Key Points:

- Ends method execution.
- Returns a value if method is not void.
- If method is declared with a type → must return that type.
- If method is void → return is optional.

CLASS STRUCTURE IN JAVA

DEFINITION

In Java, a **class** is the fundamental building block of Object-Oriented Programming. It is a **user-defined data type** that acts as a **blueprint or template** from which individual objects are created.

A class groups together **fields (variables)** and **methods (functions)** into a single unit, making it easier to represent **real-world entities** like Student, Employee, Car, etc.

Each object created from the class will have its own **copy of variables** but will share the **methods**.

OBJECT:

An object is an instance of a class. It represents a specific entity with its own values for the defined attributes.

Key points about class structure:

A class starts with the `class` keyword.

A class can contain:

- **Variables** (also called attributes, fields, or data members).
- **Methods** (functions that define the behavior of the class).
- **Constructors** (special methods used to initialize objects).
- **Blocks** (static and non-static).
- **Nested classes or interfaces**.

Syntax

```
class ClassName {
    // Fields (variables)
    datatype variableName;

    // Constructor
    ClassName(parameters) {
        // initialization code
    }

    // Methods
    returnType methodName(parameters) {
        // method body
    }
}
```

Example:

```
// Class definition
class Student {
    // Fields
    String name;
    int age;

    // Constructor
    Student(String n, int a) {
        name = n;
        age = a;
    }
```

```
    // Method
    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}


// Driver class
public class Main {
    public static void main(String[] args) {
        // Creating objects
        Student s1 = new Student("Aadil", 22);
        Student s2 = new Student("Sara", 21);

        // Calling methods
        s1.displayInfo();
        s2.displayInfo();
    }
}
```

EXPLANATION OF EXAMPLE

- Student is a class with two fields (name, age).
- The constructor initializes these fields when objects are created.
- The method displayInfo() prints student details.
- In Main, two different objects are created from the same blueprint, showing how classes provide reusability.

WHY IS CLASS STRUCTURE IMPORTANT?

- Encapsulation – groups data and methods together.
- Reusability – same class can create multiple objects.
- Abstraction – hides internal complexity, exposing only necessary details.
- Modularity – organizes code into manageable units.

MODIFIERS IN JAVA

In Java, modifiers are keywords that define the characteristics and accessibility of classes, methods, variables, and constructors. They are broadly categorized into two types: Access Modifiers and Non-Access Modifiers.

ACCESS MODIFIERS

Access modifiers control the visibility and accessibility of members (classes, methods, variables, and constructors) within a program, enforcing encapsulation. There are four types of access modifiers in Java:

- **public**: Members declared as public are accessible from anywhere in the program, including other classes and packages.
- **private**: Members declared as private are only accessible within the class in which they are declared. They cannot be accessed by other classes, even within the same package.
- **protected**: Members declared as protected are accessible within the same package and by subclasses, even if the subclasses are in different packages.
- **default** (Package-private): If no access modifier is explicitly specified, the member has default access. This means it is accessible only within the same package.

NON-ACCESS MODIFIERS

Non-access modifiers provide additional characteristics to classes, methods, and variables beyond their accessibility. Some common non-access modifiers include:

- **static:** Used for class-level members, meaning they belong to the class itself rather than to individual instances of the class.
- **final:** Used to create constants (for variables), prevent method overriding (for methods), and prevent inheritance (for classes).
- **abstract:** Used to define abstract classes and methods. Abstract methods must be implemented by concrete subclasses.
- **synchronized**: Used to control access to shared resources in multi-threaded environments, ensuring that only one thread can access a synchronized block or method at a time.
- **transient:** Used to mark fields that should not be serialized when an object is written to a persistent storage.
- **volatile:** Used to indicate that a variable's value might be modified by multiple threads, ensuring that changes to the variable are always visible to other threads.

COMPONENTS OF A JAVA CLASS

Since a Java program is a collection of objects that interact with one another, your first goal is to learn how to define the behavior of a single type of object.

In Java, the behavior of an object is determined by its class. Every class in Java has four major components:

CLASS DECLARATION

This defines the name of the class and can include modifiers (like public, abstract, final), the class keyword, the class name, and optionally, the extends keyword to indicate inheritance from a superclass and implements to indicate implementation of interfaces.

```
public class MyClass extends ParentClass implements MyInterface {
    // Class body
}
```

FIELDS

Fields are variables defined within a class that hold data for an object. Fields can be further split into two categories depending on whether the field stores data about a specific instance of a class (an object) or data about the entire class.

There are two types of fields:

- Instance Variables store information about a particular instance of a class. Also known as attributes or properties
- Class Variables store information about a whole class. Also known as static fields. If a field is declared with the static keyword, then it is a class variable, otherwise it is an instance variable

METHODS (ALSO CALLED FUNCTIONS OR SUBROUTINES)

Methods are blocks of code that define the behavior or functionality of an object. Some methods return a value, while others just perform actions.

- Signature: A method's signature includes its name and parameters.
- Return type: A method can return a value of a specific type or void if it doesn't return anything.

- Access modifiers: Like fields, methods can have different access levels, such as public, private, or protected.
- Static methods: Methods declared with the static keyword belong to the class rather than a specific instance and can be called without creating an object.

CONSTRUCTORS

A constructor is a special method used to initialize an object when it is created. It has the same name as the class and no return type. This is where instance variables are initialized and any other set up code is executed.

- Initialization: Constructors are used to set initial values for an object's instance variables.
- Default constructor: If you don't explicitly define a constructor, the Java compiler automatically provides a default, no-argument constructor.
- Constructor overloading: A class can have multiple constructors with different parameters.

You must always use the new keyword in front of every constructor call.

For example: **new Rectangle(10, 5)** creates a new Rectangle object initialized with parameters 10 and 5.

NESTED CLASSES

You can define a class within another class. This is called a nested class and helps logically group classes that are only used in one place.

- Inner classes (non-static): Associated with an instance of the outer class and can access its members, including private ones.
- Static nested classes: Not associated with an instance of the outer class and can only access the outer class's static members

```
// Class Declaration
public class Student {

    // Fields (Variables)
    String name;        // instance variable (belongs to each object)
    int age;            // instance variable
    static String college = "Kashmir University"; // class variable (shared by all objects)

    // Constructor (Default)
    Student() {
        name = "Unknown";
        age = 0;
    }

    // Constructor (Parameterized)
    Student(String n, int a) {
        name = n;
        age = a;
    }

    // Method (Instance Method)
    void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
        System.out.println("College: " + college);
    }
```

```
    // Static Method (belongs to class, not object)
    static void showCollege() {
        System.out.println("College (Static Method): " + college);
    }

    // Nested Classes
    class InnerClass {  // Inner (non-static) class
        void message() {
            System.out.println("Hello from Inner Class of " + name);
        }
    }

    static class StaticNestedClass { // Static nested class
        void message() {
            System.out.println("Hello from Static Nested Class");
        }
    }

    // Main Method to run the program
    public static void main(String[] args) {
        // Using Constructors
        Student s1 = new Student();             // default constructor
        Student s2 = new Student("Aadil", 22);   // parameterized constructor

        // Calling instance method
        s1.displayInfo();
        s2.displayInfo();

        // Calling static method
        Student.showCollege();

        // Using Inner Class
        Student.InnerClass inner = s2.new InnerClass();
        inner.message();

        // Using Static Nested Class
        Student.StaticNestedClass nested = new Student.StaticNestedClass();
        nested.message();
    }
}
```

VARIABLE DECLARATION IN A CLASS

Variables declared inside a class are called fields, or member variables. There are two main types: instance variables and class (static) variables.

INSTANCE VARIABLE DECLARATION

Instance variables define the unique state for each object created from a class. They are declared within the class but outside any method, constructor, or block.

Syntax:

```
access_modifier data_type variableName;
```

- access_modifier: Controls the visibility of the variable. Common modifiers include public, private, and protected.
- data_type: The type of data the variable can hold (e.g., int, String, double).

- variableName: A unique identifier for the variable, conventionally written in camel case (e.g., firstName, itemCount).

Example:

```
public class Book {
   private String title; // private instance variable
   public String author; // public instance variable
   protected int publicationYear; // protected instance variable
}
```

CLASS (STATIC) VARIABLE DECLARATION

Class variables are declared with the static keyword and are shared among all objects of a class. They are useful for constants or properties common to all instances.

Syntax:

```
access_modifier static data_type variableName;
```

- static: The keyword that makes the variable a class-level variable.
- final (optional): When combined with static, final creates a constant that cannot be reassigned.

Example:

```
public class MathConstants {
   public static final double PI = 3.14159; // static and final variable (constant)
   public static int bookCount = 0; // static variable shared by all instances
}
```

METHOD DECLARATION IN A CLASS

Methods define the behavior and functionality of an object. A method declaration consists of a method header and a body.

General syntax:

```
access_modifier static_or_final_modifier return_type methodName(parameter_list) {
   // method body (statements)
}
```

Key components

- access_modifier: Controls the visibility of the method (public, private, protected, or default).
- static (optional): Declares a class-level method that can be called without creating an object. It cannot access instance variables.
- final (optional): Prevents a method from being overridden by a subclass.
- return_type: The data type of the value the method returns. If no value is returned, use the void keyword.
- methodName: The name of the method, conventionally written in lower camel case (e.g., calculateArea(), getStudentName()).
- parameter_list: A comma-separated list of zero or more parameters, each with a data type and name, enclosed in parentheses ().
- method body: The code block that performs the method's task, enclosed in curly braces {}.

Example:

```
public class Calculator {
   // Method with a return type and parameters
   public int add(int number1, int number2) {
      return number1 + number2;
```

```
    }
    // Method with a void return type and a parameter
    public void printMessage(String message) {
        System.out.println(message);
    }
    // Static method
    public static void greet() {
        System.out.println("Hello, welcome to the Calculator!");
    }
}
```

CONSTRUCTOR IN JAVA

In Java, a constructor is a special method used to initialize an object when it is created. Its primary purpose is to set the initial state of a newly created object, ensuring that it is in a valid and usable condition from the start.

Constructors are automatically invoked by the Java Virtual Machine (JVM) using the new keyword during object creation, and they are essential for defining how an object's attributes should be set up.

Forgetting super() in Child Classes: Always call the parent constructor (super()) if the parent class has no default constructor, or it will lead to compilation errors.

KEY CHARACTERISTICS

- Same name as the class: A constructor's name must be identical to the class name it belongs to.
- No return type: Unlike other methods, constructors do not have a return type, not even void.
- Automatic invocation: A constructor is called implicitly when an object is instantiated.
- Initialization: It is typically used to assign initial values to an object's instance variables or to perform any startup procedures required.
- Access modifiers: Constructors can have access modifiers (public, private, protected), which control their visibility and who can create objects from that class

```
// Driver Class
class Mca {

    // Constructor
    Mca()
    {
        super();
        System.out.println("Constructor Called");
    }

    // main function
    public static void main(String[] args)
    {
        Geeks geek = new Mca();
    }
}
```

TYPES OF CONSTRUCTORS IN JAVA

Now is the correct time to discuss the types of the constructor, so primarily there are three types of constructors in Java are mentioned below:

- Default Constructor
- Parameterized Constructor
- No-Argument Constructor
- Copy Constructor

1. DEFAULT CONSTRUCTOR

A default constructor in Java is a no-argument constructor that is automatically provided by the Java compiler if no constructor is explicitly defined in the class. It initializes the object with default values (e.g., 0 for numbers, null for objects, false for booleans).

Key Points

- If you do not write any constructor in a class, the compiler automatically provides a default constructor.
- The default constructor has no parameters and an empty body.
- Once you define any constructor (parameterized or no-argument), the compiler does not provide a default constructor.
- The default constructor is mainly used to provide basic initialization of objects.

Syntax (Compiler-Generated Default Constructor)

```
class ClassName {
    // Compiler automatically generates:
    ClassName() {
        // empty body
    }
}
```

Example

```
class Student {
    int id;
    String name;

    // No constructor defined, so compiler provides default constructor
}
public class Main {
    public static void main(String[] args) {
        Student s = new Student(); // default constructor invoked
        System.out.println("ID: " + s.id);     // 0 (default value)
        System.out.println("Name: " + s.name); // null (default value)
    }
}
```

2. NO ARGUMENT CONSTRUCTOR

A no-argument constructor is a constructor that is explicitly defined by the programmer and does not take any parameters.

Unlike the default constructor (which is created automatically by the compiler), a no-argument constructor is written by the programmer.

It is often used to set custom default values for object data members instead of relying on Java's standard initialization (like `0`, `null`, `false`).

Example:

```
class Student {
   int id;
   String name;

   // Explicit no-argument constructor
   Student() {
      id = 1;              // Custom default value
      name = "Aadil";      // Custom default value
   }
}
public class Main {
   public static void main(String[] args) {
      Student s = new Student(); // No-argument constructor called
      System.out.println("ID: " + s.id);
      System.out.println("Name: " + s.name);
   }
}

Output:
ID: 1
Name: Aadil
```

3.PARAMETERIZED CONSTRUCTOR IN JAVA

A parameterized constructor is a constructor that accepts one or more arguments. It allows you to initialize an object's state with specific, user-defined values at the time of object creation.

Key Points

- A parameterized constructor takes parameters and uses them to initialize object fields.
- It provides flexibility by allowing different objects to have different initial values.
- If you define a parameterized constructor, the compiler will not provide a default constructor.
- If you still want a no-argument constructor, you must define it explicitly.

Example:

```
class Student {
   int id;
   String name;

   // Parameterized constructor
   Student(int i, String n) {
      id = i;
      name = n;
   }
}

public class Main {
   public static void main(String[] args) {
      // Passing values while creating objects
      Student s1 = new Student(101, "Aadil");
      Student s2 = new Student(102, "Rahul");

      System.out.println("ID: " + s1.id + ", Name: " + s1.name);
      System.out.println("ID: " + s2.id + ", Name: " + s2.name);
   }
}
Output:
ID: 101, Name: Aadil
```

```
ID: 102, Name: Rahul
```

CONSTRUCTOR OVERLOADING IN JAVA

Constructor Overloading in Java means defining multiple constructors in the same class, but with different parameter lists (different number of parameters or different types of parameters).

Just like method overloading, constructor overloading allows objects to be initialized in different ways depending on the constructor used.

Key Points

- Same class, same constructor name, but different parameter lists.
- Helps in flexible initialization of objects.
- Decided by the number and type of parameters passed at the time of object creation.
- If no matching constructor is found, compilation error occurs.
- Constructor overloading improves readability and makes code more versatile.

Example:

```
class Student {
   int id;
   String name;

   // No-argument constructor
   Student() {
      id = 0;
      name = "Unknown";
   }

   // Parameterized constructor (1 parameter)
   Student(int i) {
      id = i;
      name = "Not Provided";
   }

   // Parameterized constructor (2 parameters)
   Student(int i, String n) {
      id = i;
      name = n;
   }

   void display() {
      System.out.println("ID: " + id + ", Name: " + name);
   }
}

public class Main {
   public static void main(String[] args) {
      Student s1 = new Student();              // Calls no-argument constructor
      Student s2 = new Student(101);           // Calls 1-parameter constructor
      Student s3 = new Student(102, "Aadil");   // Calls 2-parameter constructor

      s1.display();
      s2.display();
      s3.display();
   }
}
```

```
Output
ID: 0, Name: Unknown
ID: 101, Name: Not Provided
ID: 102, Name: Aadil
```

COPY CONSTRUCTOR IN JAVA

A copy constructor is a type of constructor that is used to create a new object as a copy of an existing object.
It initializes the new object with the values of another object of the same class.

Unlike C++, Java does not provide a built-in copy constructor. However, we can define our own copy constructor explicitly.

Key Points

- Takes one parameter: an object of the same class.
- Used to copy the data of one object into another.
- Java does not provide it automatically; programmers must write it explicitly.
- Alternative to copy constructor: using clone() method or assignment, but copy constructor gives more control.

Example: Student Class

```
class Student {
   int id;
   String name;

   // Parameterized constructor
   Student(int i, String n) {
      id = i;
      name = n;
   }

   // Copy constructor
   Student(Student s) {
      id = s.id;       // Copying values
      name = s.name;
   }

   void display() {
      System.out.println("ID: " + id + ", Name: " + name);
   }
}

public class Main {
   public static void main(String[] args) {
      Student s1 = new Student(101, "Aadil");   // Normal constructor
      Student s2 = new Student(s1);             // Copy constructor

      s1.display();
      s2.display();
   }
}

Output
ID: 101, Name: Aadil
ID: 101, Name: Aadil
```

ADVANTAGES OF COPY CONSTRUCTOR

- Provides a simple way to duplicate objects.
- Ensures separate objects with the same data (not just references).
- Useful in cases like object backups, cloning, or passing objects safely.

Note

- If you don't explicitly write a copy constructor, you can still copy objects using assignment (=), but that only copies the reference, not the object.
- A copy constructor actually creates a new object with the same values.

CONSTRUCTOR CHAINING

Constructor chaining in Java is the process of one constructor calling another constructor.

Constructor chaining is the process of invoking a series of constructors within the same class or by the child class's constructors. In Java programming, constructor chaining generally happens as a result of the Inheritance process.

When we are chaining constructors in a Java program, the first line of a constructor should always start with either this() or super() method according to the type of constructor chaining we want to implement.

We use this() method to call a constructor from another overloaded constructor in the same class, and the super() method to invoke an immediate parent class constructor.

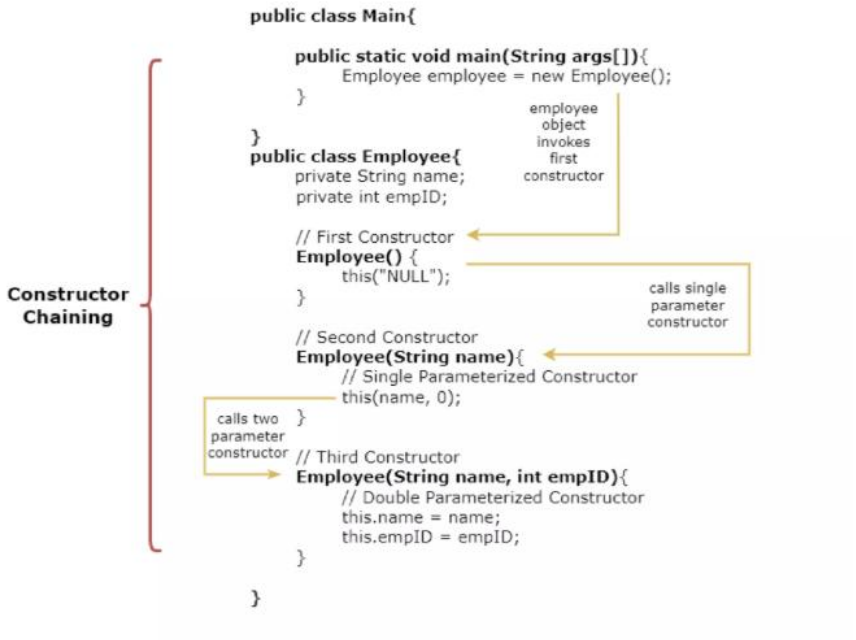
1. CONSTRUCTOR CHAINING WITHIN THE SAME CLASS:

We make several constructors with a different number of parameters (or with the different data types of parameters) within the same class and make a chain with these constructors using the this() method, we use this() method to call a constructor from another overloaded constructor in the same class. Let's see a Java example to understand it better:

Example Java Program:

```
class Main {
   public static void main(String args[]) {
      Employee employee = new Employee(); // calling the 1st constructor
      System.out.println("Employee Name: " + employee.getName());
      System.out.println("Employee ID: " + employee.getEmpID());
   }
}

class Employee{
   private String name;
   private int empID;

   // 1st constructor
   public Employee(){
      this("NULL"); // calling the 2nd constructor
   }

   // 2nd constructor
   public Employee(String name){
      this(name, 0);  // calling the 3rd constructor
   }
```

```
    // 3rd constructor
    public Employee(String name, int empID){ // fully parameterized constructor
        this.name = name;
        this.empID = empID;
    }

    public String getName(){
        return name;
    }

    public int getEmpID(){
        return empID;
    }
}
Output:
Employee Name: NULL
Employee ID: 0
```



2.CONSTRUCTOR CHAINING FROM PARENT/BASE CLASS

When extending a class in Java, constructors in the subclass can chain constructors from the base class using the super() keyword. This allows for leveraging the initialization logic defined in the superclass constructors while adding additional functionality specific to the subclass. It promotes code reuse and maintains the inheritance hierarchy.

```
class Person {
    Person() {
        System.out.println("Person class constructor called");
    }
}

class Employee extends Person {
    Employee() {
        super(); // Calls parent class constructor
        System.out.println("Employee class constructor called");
    }
```

```
}

public class Main2 {
   public static void main(String[] args) {
      Employee e = new Employee();
   }
}
```

WHY DO WE NEED FOR CONSTRUCTOR CHAINING?

- Code Reusability: Avoids duplication by reusing initialization logic across constructors.
- Consistency: Ensures all constructors share common setup steps, reducing errors.
- Simplified Maintenance: Changes to initialization logic need to be made in only one place.
- Supports Inheritance: Ensures the superclass constructor is called before subclass-specific initialization.
- Improves Readability: Cleaner and more structured code, making initialization logic easier to follow.

RULES FOR CONSTRUCTOR CHAINING

Following are the rules to keep in mind while doing Constructor Chaining:

- If an expression uses this keyword, then this() expression must be the first line of the constructor.
- When chaining constructors, the order doesn't matter.
- There must exist at least a single constructor without this keyword.

Note: We can't use this() and super() at the same time in the same constructor block/section.

FINALIZE() METHOD IN JAVA

The finalize() method in Java is a method of the Object class that the Garbage Collector (GC) calls before an object is destroyed.
It is used to perform cleanup activities (like closing files, releasing network connections, or freeing resources) before the object is removed from memory.

Syntax

```
protected void finalize() throws Throwable {
   // cleanup code
}
```

Key Points

- Defined in Object class: Every class in Java inherits it.
- Called by Garbage Collector just before destroying an object.
- Can be overridden in your class to define custom cleanup code.
- Not guaranteed to be called immediately, because Garbage Collection depends on JVM.
- From Java 9 onwards, finalize() is deprecated (not recommended) due to unpredictability. Instead, use try-with-resources or AutoCloseable.

Example

```
class Student {
   int id;
```

```
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }

    // Overriding finalize()
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Finalize called for: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Aadil");
        Student s2 = new Student(102, "Rahul");

        s1 = null; // Making object eligible for GC
        s2 = null;

        // Requesting JVM to run Garbage Collector
        System.gc();

        System.out.println("Main method finished");
    }
}

Possible Output
Main method finished
Finalize called for: Rahul
Finalize called for: Aadil
```

Note: The order and execution of finalize() depend on the JVM, so it is not predictable.

Advantages

- Provides a mechanism to release resources before an object is removed from memory.
- Useful for cleanup tasks like closing database connections or file streams.

Limitations

- Not guaranteed to be called immediately (or even at all) because Garbage Collection is uncertain.
- Deprecated from Java 9 (unreliable in production code).
- Can lead to performance issues if misused.

GARBAGE COLLECTION (GC) IN JAVA

Garbage Collection (GC) in Java is an automatic memory management process within the Java Virtual Machine (JVM). Its primary function is to reclaim memory occupied by objects that are no longer referenced by the running program, thereby preventing memory leaks and OutOfMemoryErrors.

HOW IT WORKS:

- **Automatic Memory Management**: The JVM automatically deletes unused objects.
- **Eligible for Garbage Collection**: An object becomes eligible if:
    - It is no longer referenced by any variable.

  - Its reference is set to `null`.
  - It becomes unreachable in the object graph.
- **Method `System.gc()`**: Suggests the JVM to run garbage collection, but **does not guarantee immediate execution**.
- **`finalize()` Method**: Called before an object is destroyed (can be overridden for cleanup).

HOW DOES GARBAGE COLLECTION WORK IN JAVA?

During the garbage collection process, the collector scans different parts of the heap, looking for objects that are no longer in use. If an object no longer has any references to it from elsewhere in the application, the collector removes the object, freeing up memory in the heap. This process continues until all unused objects are successfully reclaimed.

To ensure that garbage collectors work efficiently, the JVM separates the heap into separate spaces, and then garbage collectors use a mark-and-sweep algorithm to traverse these spaces and clear out unused objects. Let's take a closer look at the different generations in the memory heap.

GENERATIONAL GARBAGE COLLECTION

To fully understand how Java garbage collection works, it's important to know about the different generations in the memory heap, which help make garbage collection more efficient. These generations are split into these types of spaces:

1. ***JAVA EDEN SPACE:***

   The eden space in Java is a memory pool where objects are created. When the eden space is full, the garbage collector either removes objects if they are no longer in use or stores them in the survivor space if they are still being used. This space is considered part of the young generation in the memory heap.

2. ***JAVA SURVIVOR SPACES***

   There are two survivor spaces in the JVM: survivor zero and survivor one. This space is also part of the young generation.

3. ***JAVA TENURED SPACE***

   The tenured space is where long-lived objects are stored. Objects are eventually moved to this space if they survive a certain number of garbage collection cycles. This space is much larger than the eden space and the garbage collector checks it less often. This space is considered the old generation in the heap.

MARK-AND-SWEEP

The Java garbage collection process uses a mark-and-sweep algorithm. Here's how that works:

There are two phases in this algorithm: **mark followed by sweep.**

- When a Java object is created in the heap, it has a mark bit that is set to 0 (false).
- During the mark phase, the garbage collector traverses object trees starting at their roots. When an object is reachable from the root, the mark bit is set to 1 (true). Meanwhile, the mark bits for unreachable objects is unchanged.
- During the sweep phase, the garbage collector traverses the heap, reclaiming memory from all items with a mark bit of 0 (false).

TYPES OF GARBAGE COLLECTION

- Minor GC: Cleans the Young Generation (frequent, fast).
- Major GC / Full GC: Cleans the Old Generation (less frequent, slower).

- Stop-the-World: JVM pauses application threads during GC execution.

Example:

```
class Student implements AutoCloseable {
   int id;
   String name;

   Student(int i, String n) {
      id = i;
      name = n;
      System.out.println("Student created: " + name);
   }

   @Override
   public void close() {
      System.out.println("Cleaning up resources for: " + name);
   }
}

public class Main {
   public static void main(String[] args) {
      try (Student s1 = new Student(101, "Aadil");
           Student s2 = new Student(102, "Rahul")) {
         // Work with students
      } // close() automatically called here
      System.out.println("Main method finished");
   }
}
```

PARAMETER PASSING IN JAVA

In Java, we often pass values to methods so they can perform operations using those values. This is called parameter passing. Java supports two types of parameter passing:

- Pass-by-Value (Primitive Data Types)
- Pass-by-Reference (Objects & Arrays)

1. HOW PARAMETERS WORK IN JAVA?

In Java, all method arguments are passed by value, meaning a copy of the value is passed to the method. However, how this affects the original variable depends on whether we pass:

- Primitive data types (like int, double) → Original value does not change
- Objects & arrays → Original object can be modified

2. PASSING PRIMITIVE DATA TYPES (PASS-BY-VALUE)

When we pass a primitive data type (like int, double, char) to a method, only a copy of the value is passed. Changes inside the method do not affect the original value.

Example: Passing Primitive Data Types

```
class Example {
  // Method that tries to modify the value
  static void modifyValue(int num) {
    num = num + 10;  // Change the value
    System.out.println("Inside Method: " + num);
  }

  public static void main(String[] args) {
```

```
    int number = 50;

    // Passing primitive type to method
    modifyValue(number);

    // Original value remains unchanged
    System.out.println("Outside Method: " + number);
  }
}
```

Output:

```
Inside Method: 60
Outside Method: 50
```

Explanation: Inside the method, num is changed, but the original number remains unchanged because only a copy was passed.

3. PASSING OBJECTS (PASS-BY-REFERENCE)

When we pass an object to a method, a copy of the reference (memory address) is passed. This means changes made inside the method will affect the original object.

Example: Passing Objects to Methods

```
class Person {
  String name;

  // Constructor
  Person(String name) {
    this.name = name;
  }

  // Method that modifies the object
  static void changeName(Person p) {
    p.name = "Updated Name";  // Modifying object
  }

  public static void main(String[] args) {
    // Creating an object
    Person person1 = new Person("John");

    // Printing before modification
    System.out.println("Before: " + person1.name);

    // Passing object to method
    changeName(person1);

    // Printing after modification
    System.out.println("After: " + person1.name);
  }
}
```

Output:

```
Before: John
After: Updated Name
```

Explanation: The Person object is passed to the method, and its name property is changed.

Since objects are passed by reference, the original object is modified.

4. PASSING ARRAYS TO METHODS

Like objects, arrays are also passed by reference, meaning changes inside the method will affect the original array.

Example: Passing an Array to a Method

```
class Example {
  // Method that modifies an array
  static void modifyArray(int[] arr) {
    arr[0] = 100;  // Changing the first element
  }

  public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4};

    // Printing before modification
    System.out.println("Before: " + numbers[0]);

    // Passing array to method
    modifyArray(numbers);

    // Printing after modification
    System.out.println("After: " + numbers[0]);
  }
}
```

Output:

```
Before: 1
After: 100
```

Explanation: The modifyArray() method changes the first element of the array.

Since arrays are passed by reference, the original array is modified.

WHAT IS VARIABLE HIDING IN JAVA?

Variable hiding occurs when a subclass declares a variable with the same name as a variable in its superclass. The subclass's variable hides the one in the superclass.

Example: Variable Hiding (Instance Variables)

Scenario: A Child class extends a Parent class. Both define a variable name.

```
public class Main {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.printNames();
    }
}

class Parent {
    String name = "ParentClass";
}

class Child extends Parent {
    String name = "ChildClass"; // hides Parent's 'name'

    void printNames() {
        System.out.println("Child name: " + name);        // accesses Child's 'name'
        System.out.println("Parent name: " + super.name); // accesses Parent's 'name'
    }
}
```

Output:

```
Child name: ChildClass
Parent name: ParentClass
```

Explanation: This code demonstrates variable hiding in Java, where the Child class defines a variable name that hides the same variable from the Parent class. Using super.name, the child can still access the hidden parent variable.

STATIC VARIABLE HIDING

When a static variable with the same name is declared in a subclass, it hides the static variable in the superclass.

Example: Both Parent and Child classes define a static variable id.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Child.id: " + Child.id);     // prints Child's static id
        System.out.println("Parent.id: " + Parent.id);   // prints Parent's static id
    }
}

class Parent {
    static String id = "1";
}

class Child extends Parent {
    static String id = "2"; // hides Parent's static 'id'
}
Output:
Child.id: 2
Parent.id: 1
```

Explanation: This example shows static variable hiding, where Child defines a static variable id that hides the id in Parent. Accessing Child.id and Parent.id directly shows how each class maintains its own version of the static variable.

METHOD OVERLOADING

In Java, Method Overloading allows us to define multiple methods with the same name but different parameters within a class.

Method overloading in Java is also known as **Compile-time Polymorphism, Static Polymorphism, or Early binding,** because the decision about which method to call is made at compile time based on the method's signature.

Method overloading allows you to perform similar operations with varying inputs using a single, intuitive method name, making the code more readable and easier to maintain.

```
public class Sum {

    // Overloaded sum()
    // This sum takes two int parameters
    public int sum(int x, int y) { return (x + y); }

    // Overloaded sum()
    // This sum takes three int parameters
    public int sum(int x, int y, int z)
    {
        return (x + y + z);
    }

    // Overloaded sum()
    // This sum takes two double parameters
```

```
    public double sum(double x, double y)
    {
        return (x + y);
    }

    // Driver code
    public static void main(String args[])
    {
        Sum s = new Sum();
        System.out.println(s.sum(10, 20));
        System.out.println(s.sum(10, 20, 30));
        System.out.println(s.sum(10.5, 20.5));
    }
}
```

DIFFERENT WAYS OF METHOD OVERLOADING IN JAVA

The different ways of method overloading in Java are mentioned below:

1. CHANGING THE NUMBER OF PARAMETERS

Method overloading can be achieved by changing the number of parameters while passing to different methods.

2. CHANGING DATA TYPES OF THE ARGUMENTS

In many cases, methods can be considered overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

3. CHANGING THE ORDER OF THE PARAMETERS OF METHODS

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods.

WHAT IF THE EXACT PROTOTYPE DOES NOT MATCH WITH ARGUMENTS?

- If no method matches exactly, priority-wise, the compiler takes these steps:
- Type Conversion but to a higher type (in terms of range) in the same family (for example, byte -> int)
- Type conversion to the next higher family. Suppose if there is no long data type available for an int data type, then it will search for the float data type.
- If no suitable method is found, compilation error.

THIS KEYWORD

In Java, this is a reference variable that refers to the current object of a class. Whenever an object is created, a reference to that object is stored in this.

USES OF THIS KEYWORD

1. TO REFER TO CURRENT INSTANCE VARIABLES

When local variables (method parameters) and instance variables have the same name, this is used to avoid ambiguity.

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
```

```
        this.name = name; // "this" refers to current object's variable
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + this.name + ", Age: " + this.age);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("jhon", 20);
        s1.display();
    }
}
```

Output:

Name: jhon, Age: 20

---

2. TO CALL ANOTHER CONSTRUCTOR IN THE SAME CLASS

We can use this() to perform constructor chaining.

3. TO CALL CURRENT CLASS METHODS

You can use this to call a method of the current class (useful when methods are overloaded).

```
class Calculator {
    void display() {
        System.out.println("No parameters");
    }

    void display(int x) {
        this.display(); // Calls display() method
        System.out.println("Value: " + x);
    }
}

public class Main3 {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        c.display(10);
    }
}
```

Output:

No parameters
Value: 10

4. TO PASS CURRENT OBJECT AS A PARAMETER

this can be passed as an argument to another method or constructor.

```
class A {
    void display(A obj) {
        System.out.println("Method called using this");
    }

    void call() {
        display(this); // Passing current object
    }
}
```

```
public class Main4 {
    public static void main(String[] args) {
        A obj = new A();
        obj.call();
    }
}
Output:
Method called using this
```

5. TO RETURN CURRENT OBJECT FROM A METHOD

This is often used in method chaining.

```
class Student {
    String name;

    Student setName(String name) {
        this.name = name;
        return this; // returning current object
    }

    void show() {
        System.out.println("Name: " + name);
    }
}

public class Main5 {
    public static void main(String[] args) {
        new Student().setName("Aadil").show(); // method chaining
    }
}
Output:
Name: Aadil
```

KEY POINTS TO REMEMBER

- this always refers to the current object.
- Can be used to remove ambiguity between local and instance variables.
- Can call constructors and methods of the current class.
- Can pass the current object as a parameter or return it.
- Cannot be used inside static methods, because static methods belong to the class, not to an object.

INHERITANCE IN JAVA

Inheritance in Java is an object-oriented programming mechanism where one class, called the child class (subclass or derived class), acquires the fields (data members) and methods (functions) of another class, called the parent class (superclass or base class).

This means that the child class can reuse the code defined in the parent class, extend it with new features, or override the parent's methods to provide its own implementation.

In other words, inheritance establishes an "IS-A" relationship between classes, where the subclass is a specialized version of the superclass. For example:

- A Car is a Vehicle

- A Dog is an Animal

WHY USE INHERITANCE IN JAVA?

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.
- **Method Overriding:** Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.
- **Abstraction:** The concept of abstraction where we do not have to provide all details, is achieved through inheritance. Abstraction only shows the functionality to the user.

KEY TERMINOLOGIES USED IN JAVA INHERITANCE

- **Class:** Class is a set of objects that share common characteristics/ behavior and common properties/ attributes. Class is not a real-world entity. It is just a template or blueprint or prototype from which objects are created.
- **Super Class/Parent Class**: The class whose features are inherited is known as a superclass(or a base class or a parent class).
- **Sub Class/Child Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Extends Keyword:** This keyword is used to inherit properties from a superclass.

TYPES OF INHERITANCE IN JAVA

Below are the different types of inheritance which are supported by Java.

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance
- Hybrid Inheritance

1. SINGLE INHERITANCE

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behaviour of a single-parent class. Sometimes, it is also known as simple inheritance.

```
//Super class
class Vehicle {
   Vehicle() {
      System.out.println("This is a Vehicle");
   }
}

// Subclass
class Car extends Vehicle {
   Car() {
      System.out.println("This Vehicle is Car");
   }
}

public class Test {
   public static void main(String[] args) {
      // Creating object of subclass invokes base class constructor
      Car obj = new Car();
   }
}
```

Output:
This is a vehicle
This Vehicle is Car

2. MULTILEVEL INHERITANCE

In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also acts as the base class for other classes.

```
class Vehicle {
   Vehicle() {
      System.out.println("This is a Vehicle");
   }
}
class FourWheeler extends Vehicle {
   FourWheeler() {
      System.out.println("4 Wheeler Vehicles");
   }
}
class Car extends FourWheeler {
   Car() {
      System.out.println("This 4 Wheeler Vehicle is a Car");
   }
}
public class Geeks {
   public static void main(String[] args) {
      Car obj = new Car(); // Triggers all constructors in order
   }
}
```

3. HIERARCHICAL INHERITANCE

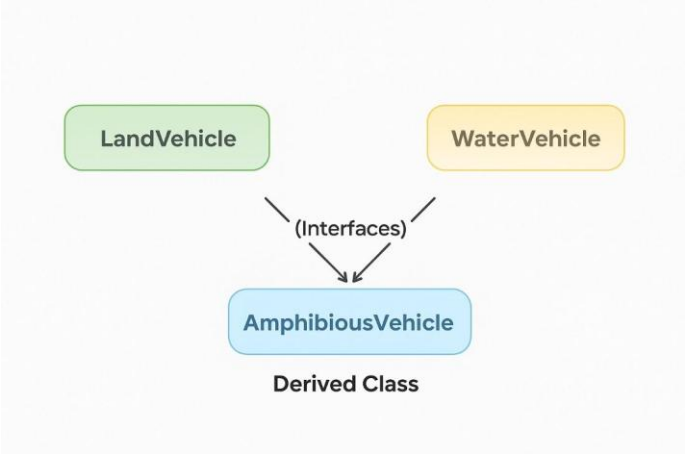
In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class. For example, cars and buses both are vehicle

```
class Vehicle {
   Vehicle() {
      System.out.println("This is a Vehicle");
   }
}

class Car extends Vehicle {
   Car() {
      System.out.println("This Vehicle is Car");
   }
}

class Bus extends Vehicle {
   Bus() {
      System.out.println("This Vehicle is Bus");
   }
}

public class Test {
   public static void main(String[] args) {
      Car obj1 = new Car();
      Bus obj2 = new Bus();
   }
}
```

4. MULTIPLE INHERITANCE (THROUGH INTERFACES)

In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes.

Note: that Java does not support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces.




```
interface LandVehicle {
   default void landInfo() {
      System.out.println("This is a LandVehicle");
   }
}
interface WaterVehicle {
   default void waterInfo() {
      System.out.println("This is a WaterVehicle");
   }
}
// Subclass implementing both interfaces
class AmphibiousVehicle implements LandVehicle, WaterVehicle {
   AmphibiousVehicle() {
      System.out.println("This is an AmphibiousVehicle");
   }
}
public class Test {
   public static void main(String[] args) {
      AmphibiousVehicle obj = new AmphibiousVehicle();
      obj.waterInfo();
      obj.landInfo();
   }
}
```

5. HYBRID INHERITANCE

It is a mix of two or more of the above types of inheritance. In Java, we can achieve hybrid inheritance only through Interfaces if we want to involve multiple inheritance to implement Hybrid inheritance.

ROLE OF ACCESS MODIFIERS IN INHERITANCE IN JAVA

Access modifiers in Java define how members (fields, methods, constructors) of a class can be accessed in other classes, especially when inheritance is involved. There are four main access modifiers in Java:

1. PUBLIC

Members declared as public are inherited everywhere.

Accessible inside the same class, same package, different package, and by subclasses.

```
class Parent {
   public void show() {
      System.out.println("Public method of Parent");
   }
}

class Child extends Parent {}

public class Main {
   public static void main(String[] args) {
      Child c = new Child();
      c.show(); // ✓ Accessible
   }
}
```

Output:

```
Public method of Parent
```

2. PROTECTED

Members declared as protected are inherited in the same package and also accessible in subclasses even if they are in different packages.

Outside package → only accessible through inheritance, not by object reference.

```
class Parent {
   protected void display() {
      System.out.println("Protected method of Parent");
   }
}

class Child extends Parent {
   void callMethod() {
      display(); // ✓ Accessible through inheritance
   }
}

public class Main {
   public static void main(String[] args) {
      Child c = new Child();
      c.callMethod();
      // c.display(); ✗ Not directly accessible outside package
   }
}
```

---

3. DEFAULT (NO MODIFIER)

Also called package-private.

Members declared with no modifier are inherited only within the same package.

Not accessible in subclasses outside the package.

```
class Parent {
   void message() { // default
      System.out.println("Default method of Parent");
```

```
    }
}

class Child extends Parent {}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.message(); // ✓ Accessible (same package)
    }
}
```

4. PRIVATE

Members declared as private are NOT inherited.

They are accessible only within the same class.

If a child needs access, the parent must provide getter/setter methods.

```
class Parent {
    private int value = 10;

    private void secret() {
        System.out.println("Private method of Parent");
    }

    public int getValue() { // ✓ Accessible through getter
        return value;
    }
}

class Child extends Parent {
    void show() {
        // secret(); ✗ Not accessible
        System.out.println("Value from Parent: " + getValue()); // ✓ via getter
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
    }
}
Output:
Value from Parent: 10
```

METHOD OVERRIDING IN JAVA

Method Overriding in Java is an object-oriented programming feature that allows a subclass (child class) to provide its own specific implementation of a method that is already defined in its superclass (parent class).

In this case, the method in the child class has the same name, return type, and parameters as the method in the parent class, but its body is redefined to perform a different task.

In simple terms:

Overriding = Same method name + Same parameters + Different implementation in child class.

It represents Runtime Polymorphism (Dynamic Method Dispatch) because the method that gets executed is determined at runtime depending on the object type.

RULES FOR OVERRIDING:

- Name, parameters, and return type must match the parent method.
- Java picks which method to run, based on the actual object type, not just the variable type.
- Static methods cannot be overridden.
- The @Override annotation catches mistakes like typos in method names.

```
class Animal {
   void sound() {
      System.out.println("This animal makes a sound");
   }
}

class Dog extends Animal {
   @Override
   void sound() {
      System.out.println("The dog barks");
   }
}
```

In this example, the sound() method is defined in both the Animal superclass and the Dog subclass. The Dog class provides its specific implementation of the sound() method.

EXAMPLE WITH SUPER KEYWORD

The super keyword is often used in method overriding to call the superclass's version of a method. This is useful when you want to retain the behavior of the parent class's method while adding additional functionality in the subclass.

```
class Animal {
   void sound() {
      System.out.println("This animal makes a sound");
   }
}

class Dog extends Animal {
   @Override
   void sound() {
      super.sound();  // Calls the superclass method
      System.out.println("The dog barks");
   }
}
public class Main {
   public static void main(String[] args) {
      Dog myDog = new Dog();
      myDog.sound();  // Calls the overridden method
   }
}
```

Here, the Dog class overrides the sound() method but uses the super.sound() call to invoke the original behavior from the Animal class before adding its own behavior.

METHOD OVERRIDING AND ACCESS MODIFIERS

- Public methods can be overridden, but they must remain public or more accessible in the subclass.
- Protected methods can also be overridden, and they can be kept protected or made public.
- Private methods are not visible to the subclass and cannot be overridden.

ADVANTAGES

- Provides runtime polymorphism.
- Helps in implementing the concept of "programming to an interface."
- Encourages code reuse by allowing existing methods to be modified for specific needs.

METHOD SHADOWING

When superclass and subclass contain the same method including parameters and if they are static. The method in the superclass will be hidden by the one that is in the subclass. This mechanism is known as method shadowing.

```
class Super{
  public static void demo() {
    System.out.println("This is the main method of the superclass");
  }
}
class Sub extends Super{
  public static void demo() {
    System.out.println("This is the main method of the subclass");
  }
}

public class MethodHiding{
  public static void main(String args[]) {
    MethodHiding obj = new MethodHiding();
    Sub.demo();
  }
}
```

KEY CHARACTERISTICS OF METHOD SHADOWING:

- Applies to static methods: Method shadowing specifically applies to static methods, not instance methods. For instance methods, the concept is method overriding.
- Compile-time polymorphism: Unlike method overriding, which exhibits runtime polymorphism, method shadowing is resolved at compile time. The method that is invoked depends on the type of the reference variable, not the actual object it refers to.
- No dynamic dispatch: Since static methods belong to the class and not to an object, there is no dynamic dispatch based on the object's actual type at runtime.

USE OF SUPER KEYWORD.

The super keyword in Java is used to refer to the immediate parent class object. It is commonly used to access parent class methods and constructors, enabling a subclass to inherit and reuse the functionality of its superclass.

Usage: The super keyword can be used in three primary contexts:

- To call the superclass constructor.
- To access a method from the superclass that has been overridden in the subclass.
- To access a field from the superclass when it is hidden by a field of the same name in the subclass.

Syntax

```
super();
super.methodName();
```

```
super.fieldName;
```

Examples

EXAMPLE 1: CALLING SUPERCLASS CONSTRUCTOR

```
class Animal {
   Animal() {
      System.out.println("Animal constructor called");
   }
}

class Dog extends Animal {
   Dog() {
      super(); // Calls the constructor of Animal class
      System.out.println("Dog constructor called");
   }
}

public class TestSuper {
   public static void main(String[] args) {
      Dog d = new Dog(); // Output: Animal constructor called
                         //         Dog constructor called
   }
}
```

In this example, the super() call in the Dog constructor invokes the constructor of the Animal class.

EXAMPLE 2: ACCESSING SUPERCLASS METHOD

```
class Animal {
   void sound() {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal {
   void sound() {
      super.sound(); // Calls the sound() method of Animal class
      System.out.println("Dog barks");
   }
}

public class TestSuper {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.sound(); // Output: Animal makes a sound
                 //         Dog barks
   }
}
```

Here, the super.sound() call in the Dog class method sound() invokes the sound() method of the Animal class.

EXAMPLE 3: ACCESSING SUPERCLASS FIELD

```
class Animal {
   String color = "white"; // Parent class property
}

class Dog extends Animal {
   String color = "black"; // Child class property

   void displayColor() {
```

```
        System.out.println("Dog color: " + color);        // prints Dog's color
        System.out.println("Animal color: " + super.color); // prints Animal's color using super
    }
}

public class TestSuper {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.displayColor();
    }
}
```

In this example, super.color is used to access the color field of the Animal class, which is hidden by the color field in the Dog class.

POLYMORPHISM

Polymorphism, a core principle of Object-Oriented Programming (OOP) in Java, allows an object to take on many forms. The word "polymorphism" originates from Greek, where "poly" means "many" and "morphs" means "forms." In Java, this means a single action can be performed in different ways, depending on the object type involved.

TYPES OF POLYMORPHISM

There are two main types of polymorphism in Java: compile-time and runtime.

COMPILE-TIME POLYMORPHISM (EARLY BINDING)

Compile-time polymorphism is also called static polymorphism. It allows users to define multiple methods with the same name but different parameters. It is beneficial because it helps write cleaner and more efficient code.

The two types of compile-time polymorphism are:

1. **Method Overloading:** In method overloading, the methods with the same name can have different numbers of parameters, different types of parameters, or different order of parameters in methods.
2. **Operator Overloading:** It is the process of providing multiple meanings to an operator. Java offers limited support for in-built operator overloading. The '+' operator is used for addition of numbers and concatenation of strings. It is important to note that user-defined operator overloading is not supported.

```
public class OperatorOverloadingExample {
        public static void main(String[] args) {
          // Operator '+' used for addition
          int a = 10;
          int b = 20;
          int sum = a + b;
        System.out.println("Sum of a and b is: " + sum);

        // Operator '+' used for String concatenation
        String firstName = "Java";
        String lastName = "Programming";
        String fullName = firstName + " " + lastName;
        System.out.println("Full Name: " + fullName);
    }
}
```

RUNTIME POLYMORPHISM

Runtime polymorphism is also known as dynamic method dispatch. This process handles the call to an overridden method dynamically during runtime.

The following is a type of runtime polymorphism in Java.

Method Overriding: It is a type of runtime polymorphism where the method in the subclass overrides the method in the superclass, which has the same name, parameters, and return type.

ADVANTAGES OF POLYMORPHISM

Polymorphism provides many advantages that can improve the readability, maintainability, and efficiency of code, such as:

Code Reusability, Flexibility, Reduced Complexity, Simplified Coding, Better Organization

DISADVANTAGES OF POLYMORPHISM IN JAVA

Here are some potential disadvantages of polymorphism:

Complexity, Debugging, Performance Overhead, Overuse, Name Ambiguity

ABSTRACTION

Data abstraction is the process of hiding certain details and showing only essential information to the user. Abstraction can be achieved with either abstract classes or interfaces.

ABSTRACT CLASS

An abstract class is a class declared with the abstract keyword. It serves as a blueprint for other classes and cannot be instantiated directly, meaning you cannot create an object of an abstract class.

Here are the key characteristics of abstract classes in Java:

Declaration: An abstract class is declared using the abstract keyword in its class definition.

```
abstract class MyAbstractClass {
    // ...
}
```

Instantiation: You cannot create an object of an abstract class using the new keyword.

Abstract Methods: An abstract class can contain abstract methods, which are methods declared without an implementation (no method body) and end with a semicolon, and are created using abstract keyword. Any subclass extending an abstract class must provide an implementation for all inherited abstract methods, or it too must be declared abstract.

```
abstract class Shape {
    abstract void draw(); // Abstract method
    // ...
}
```

- Abstract class can contain the normal methods
- Abstract classes can have constructors, static methods, and final methods.

Purpose: Abstract classes are used to achieve abstraction, a core OOP principle. They allow you to define a common interface and some shared functionality for a group of related classes, while deferring the specific implementation details of certain methods to the subclasses. This promotes code reusability and enforces a contract for subclasses to follow.

Example:

```
abstract class Vehicle {
   abstract void start(); // Abstract method
   void accelerate() {
      System.out.println("Vehicle accelerating...");
   }
}

class Car extends Vehicle {
   @Override
   void start() {
      System.out.println("Car starting with key.");
   }
}

public class Main {
   public static void main(String[] args) {
      // Vehicle myVehicle = new Vehicle(); // Error: Cannot instantiate abstract class
      Car myCar = new Car();
      myCar.start();
      myCar.accelerate();
   }
}
```

JAVA INTERFACES

An interface is a completely "abstract class" that is used to group related methods with empty bodies. Java interface is a collection of abstract methods. The interface is used to achieve abstraction in which you can define methods without their implementations (without having the body of the methods). An interface is a reference type and is similar to the class.

Along with abstract methods, an interface may also contain constants, default methods, static methods, and nested types. Method bodies exist only for default methods and static methods.

```
// interface
interface Animal {
  public void animalSound(); // interface method (does not have a body)
  public void run(); // interface method (does not have a body)
}
```

To access the interface methods, the interface must be "implemented" (kinda like inherited) by another class with the implements keyword (instead of extends). The body of the interface method is provided by the "implement" class:

Example

```
// Interface
interface Animal {
  public void animalSound(); // interface method (does not have a body)
}

// Pig "implements" the Animal interface
class Pig implements Animal {
  public void animalSound() {
    // The body of animalSound() is provided here
    System.out.println("The pig says: wee wee");
  }
}

class Main {
```

```
  public static void main(String[] args) {
    Pig myPig = new Pig();  // Create a Pig object
    myPig.animalSound();
  }
}
```

NOTES ON INTERFACES:

- Like abstract classes, interfaces cannot be used to create objects
- Interface methods do not have a body - the body is provided by the "implement" class
- On implementation of an interface, you must override all of its methods
- Interface methods are by default abstract and public
- Interface attributes are by default public, static and final
- An interface cannot contain a constructor (as it cannot be used to create objects)

WHY AND WHEN TO USE INTERFACES?

1. To achieve security - hide certain details and only show the important details of an object (interface).
2. Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can implement multiple interfaces. Note: To implement multiple interfaces, separate them with a comma (see example below).

MULTIPLE INTERFACES

To implement multiple interfaces, separate them with a comma:

```
interface FirstInterface {
  public void myMethod(); // interface method
}

interface SecondInterface {
  public void myOtherMethod(); // interface method
}

class DemoClass implements FirstInterface, SecondInterface {
  public void myMethod() {
    System.out.println("Some text..");
  }
  public void myOtherMethod() {
    System.out.println("Some other text...");
  }
}
class Main {
  public static void main(String[] args) {
    DemoClass myObj = new DemoClass();
    myObj.myMethod();
    myObj.myOtherMethod();
  }
}
```

EXCEPTION HANDLING IN JAVA

Exception handling in Java is a mechanism to manage runtime errors and maintain the normal flow of an application. It prevents program termination due to unexpected events. Java uses keywords like try, catch, finally, throw, and throws for this purpose.

Key Concepts:

- **Exceptions:** Events that disrupt the normal flow of a program. They are objects that are thrown at runtime.
- **try block:** Contains the code that might throw an exception.
- catch block: Follows a try block and contains the code to handle a specific type of exception.
- **finally block:** An optional block that always executes, regardless of whether an exception occurred or was caught. It's often used for resource cleanup.
- **throw keyword:** Used to explicitly throw an exception object.
- **throws keyword:** Used in a method signature to declare the types of exceptions that a method might throw, especially for checked exceptions.

EXCEPTION OBJECT

In Java, an **Exception object** is an instance of a class that represents an error or unusual event that happens while a program is running and interrupts its normal flow. All exceptions come from the **java.lang.Throwable** class, with **java.lang.Exception** used mainly for errors that a program can handle and recover from.

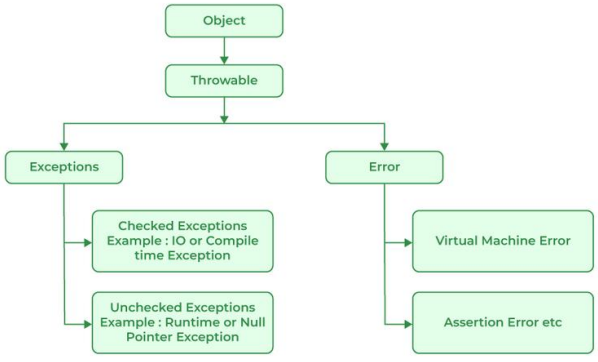
This base class has two main child classes:

- class Exception: base class for most exception types
- class Error: class that represents things that are usually not recoverable (e.g. VirtualMachineError)

JAVA EXCEPTION HIERARCHY

In Java, all exceptions and errors are subclasses of the Throwable class. It has two main branches

- Exception.
- Error




```
import java.io.*;
class Geeks {
   public static void main(String[] args)
   {
      int n = 10;
      int m = 0;

      try {

         // Code that may throw an exception
         int ans = n / m;
         System.out.println("Answer: " + ans);
      }
      catch (ArithmeticException e) {
```

```
            // Handling the exception
            System.out.println(
                "Error: Division by zero is not allowed!");
        }
        finally {
            System.out.println(
                "Program continues after handling the exception.");
        }
    }
}
#without handling the exception it would result in java.lang.ArithmeticException: / by zero
```

MAJOR REASONS WHY AN EXCEPTION OCCURS

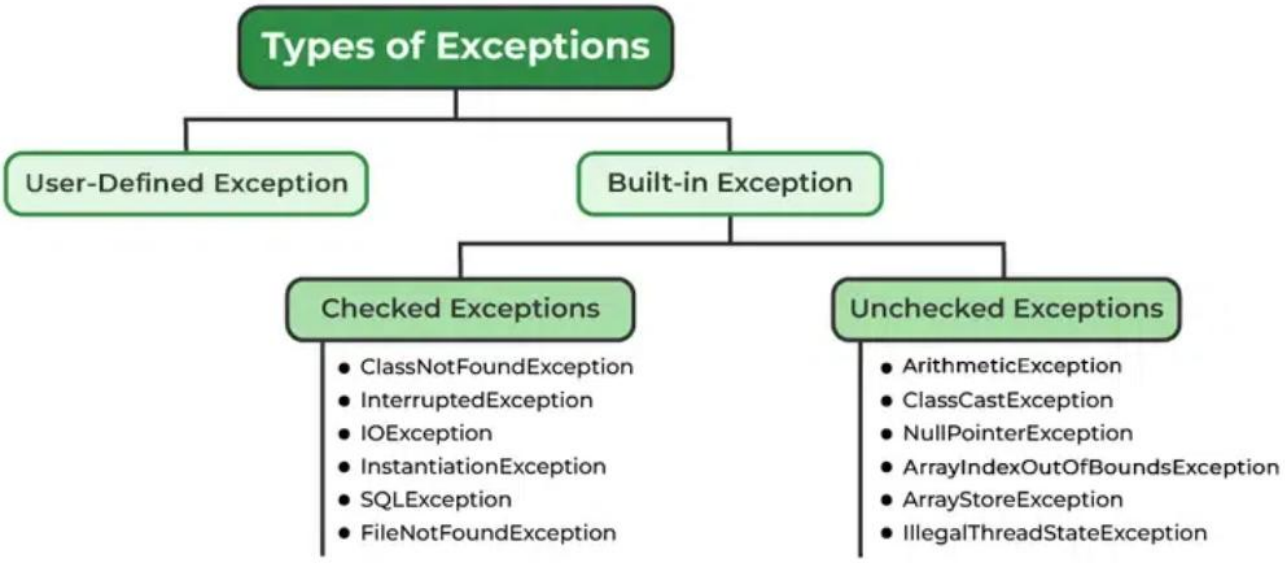
Exceptions can occur due to several reasons, such as:

- Invalid user input
- Device failure
- Loss of network connection
- Out of bound
- Null reference
- Arithmetic errors

Errors are usually beyond the control of the programmer and we should not try to handle errors.

TYPES OF JAVA EXCEPTIONS

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their it's exceptions.



Exceptions can be categorized in two ways:

1. BUILT-IN EXCEPTIONS

- Checked Exception
- Unchecked Exception

2. USER-DEFINED EXCEPTIONS

1. BUILT-IN EXCEPTION

Build-in Exception are pre-defined exception classes provided by Java to handle common errors during program execution. There are two type of built-in exception in java.

CHECKED EXCEPTIONS

Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler. Examples of Checked Exception are listed below:

- ClassNotFoundException: Throws when the program tries to load a class at runtime but the class is not found because it's belong not present in the correct location or it is missing from the project.
- InterruptedException: Thrown when a thread is paused and another thread interrupts it.
- IOException: Throws when input/output operation fails.
- InstantiationException: Thrown when the program tries to create an object of a class but fails because the class is abstract, an interface or has no default constructor.
- SQLException: Throws when there is an error with the database.
- FileNotFoundException: Thrown when the program tries to open a file that does not exist.

UNCHECKED EXCEPTIONS

Unchecked exceptions are the opposite of checked exceptions. The compiler does **not** check them at compile time. In simple terms, if a program throws an unchecked exception and you don't handle or declare it, the program will still compile without errors. Examples of Unchecked Exception are listed below:

- ArithmeticException: It is thrown when there is an illegal math operation.
- ClassCastException: It is thrown when we try to cast an object to a class it does not belong to.
- NullPointerException: It is thrown when we try to use a null object (e.g. accessing its methods or fields).
- ArrayIndexOutOfBoundsException: This occurs when we try to access an array element with an invalid index.
- ArrayStoreException: This happens when we store an object of the wrong type in an array.
- IllegalThreadStateException: It is thrown when a thread operation is not allowed in its current state.

2. USER-DEFINED EXCEPTION

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called "user-defined Exceptions".

CATCHING AN EXCEPTION: TRY-CATCH BLOCK

A try-catch block in Java is a mechanism to handle exception. The try block contains code that might throw an exception and the catch block is used to handle the exceptions if it occurs.

Internal working of try-catch Block

- Java Virtual Machine starts executing the code inside the try block.
- If an exception occurs, the remaining code in the try block is skipped and the JVM starts looking for the matching catch block.
- If a matching catch block is found, the code in that block is executed.
- After the catch block, control moves to the finally block (if present).

- If no matching catch block is found the exception is passed to the JVM default exception handler.
- The final block is executed after the try catch block. regardless of whether an exception occurs or not.

```
try {
   // Code that may throw an exception
} catch (ExceptionType e) {
   // Code to handle the exception
}
```

FINALLY BLOCK

The finally block is used to execute important code regardless of whether an exception occurs or not.

Note: finally block is always executes after the try-catch block. It is also used for resource cleanup.

```
try {
   // Code that may throw an exception
} catch (ExceptionType e) {
   // Code to handle the exception
}finally{
// cleanup code
}
```

HANDLING MULTIPLE EXCEPTION

We can handle multiple type of exceptions in Java by using multiple catch blocks, each catching a different type of exception.

```
try {
   // Code that may throw an exception
} catch (ArithmeticException e) {
   // Code to handle the exception
} catch(ArrayIndexOutOfBoundsException e){
   //Code to handle the anothert exception
}catch(NumberFormatException e){
    //Code to handle the anothert exception
}
```

THROW AND THROWS IN JAVA

THROWS

throws is a keyword in Java that is used in the signature of a method to indicate that this method might throw one of the listed type exceptions. The caller to these methods has to handle the exception using a try-catch block.

Syntax:

```
accessModifier returnType methodName() throws ExceptionType1, ExceptionType2 … {
  // code
}
```

As you can see from the above syntax, we can use throws to declare multiple exceptions.

```
public class Main {
  static void checkAge(int age) throws ArithmeticException {
    if (age < 18) {
```

```
      throw new ArithmeticException("Access denied - You must be at least 18 years old.");
    }
    else {
      System.out.println("Access granted - You are old enough!");
    }
  }

  public static void main(String[] args) {
    checkAge(15); // Set age to 15 (which is below 18...)
  }
}
```

THROW

The throw keyword in Java is used to explicitly throw an exception from a method or any block of code. We can throw either checked or unchecked exception. The throw keyword is mainly used to throw custom exceptions.

Syntax:

```
throw throwableObject;
```

A throwable object is an instance of class Throwable or subclass of the Throwable class.

Example:

```
class Main {
  public static void divideByZero() {
    throw new ArithmeticException("Trying to divide by 0");
  }
  public static void main(String[] args) {
    divideByZero();
  }}
```

The flow of execution of the program stops immediately after the throw statement is executed and the nearest enclosing try block is checked to see if it has a catch statement that matches the type of exception. If it finds a match, control is transferred to that statement otherwise next enclosing try block is checked and so on. If no matching catch is found then the default exception handler will halt the program.

DIFFERENCE BETWEEN THROW AND THROWS

The main differences between throw and throws in Java are as follows:

throw	**throws**
It is used to explicitly throw an exception	It is used to declare that a method might throw one or more exceptions
It is used inside a method or a block of code	It is used in the method signature
It can throw both checked and unchecked exceptions	It is only used for checked exceptioins. Unchecked exceptions do not require throws
The method or block throws the exception	The methods caller is responsible for handling the exception
Stops the current flow of execution immediately	It forces the caller to handle the declared exceptions.
Throw new ArithmeticException(“Error”)	Public void myMethod() throws IOException{}

CUSTOM EXCEPTIONS

Java provides us the facility to create our own exceptions by extending the Java Exception class. Creating our own Exception is known as a custom exception in Java or a user-defined exception in Java. In simple words, we can say that a User-Defined Custom Exception or custom exception is creating your own exception class and throwing that exception using the "throw" keyword.

Example: In this example, a custom exception MyException is created and thrown in the program.

```
// A Class that represents user-defined exception
class MyException extends Exception {
    public MyException(String m) {
        super(m);
    }
}

// A Class that uses the above MyException
public class setText {
    public static void main(String args[]) {
        try {

            // Throw an object of user-defined exception
            throw new MyException("This is a custom exception");
        }
        catch (MyException ex) {
            System.out.println("Caught");
            System.out.println(ex.getMessage());
        }
    }
}
```

CHAINED EXCEPTIONS IN JAVA

Chained exceptions in Java allow one exception to be linked (or chained) to another exception. This is useful when the original cause of an error is different from the exception you want to throw, but you still want to keep track of the root cause.

For example:

Suppose a method tries to read data from a file.

- If the file is corrupted, it may throw an IOException.
- But in your program, you might want to wrap this into a CustomException while still remembering that the real cause was an IOException.

Java supports this through constructors and methods of the Throwable class:

- Throwable(Throwable cause)
- Throwable(String message, Throwable cause)
- getCause() → returns the original cause.
- initCause(Throwable cause) → initializes the cause later.

Example:

```
public class ChainedExample {
    public static void main(String[] args) {
        try {
            try {
                int a = 10 / 0; // This will cause ArithmeticException
```

```
            } catch (ArithmeticException e) {
                // Wrap ArithmeticException inside a new Exception
                throw new Exception("Something went wrong", e);
            }
        } catch (Exception ex) {
            // Printing top-level exception
            System.out.println("Caught: " + ex.getMessage());
            // Printing the original cause
            System.out.println("Cause: " + ex.getCause());
        }
    }
}
#output:
Caught: Something went wrong
Cause: java.lang.ArithmeticException: / by zero
```

PACKAGES IN JAVA

A package in Java is a collection of related classes, interfaces, and sub-packages. It is used to group classes logically, similar to how folders are used in a computer to organize files.

Key Points:

- Helps in code reusability.
- Avoids name conflicts (e.g., java.util.Date vs java.sql.Date).
- Provides access protection (with access modifiers).
- Makes project management easier.

Example:

- java.util → contains utility classes like ArrayList, HashMap.
- java.io → contains classes for input-output operations.

By grouping related classes into packages, Java promotes data encapsulation, making code reusable and easier to manage. Simply import the desired class from a package to use it in your program.

TYPES OF PACKAGES:

1. ***BUILT-IN PACKAGES:***

   These are pre-defined packages provided by the Java API, such as java.lang (fundamental classes), java.util (utility classes), java.io (input/output operations), java.net (networking), and java.sql (database connectivity).

2. ***USER-DEFINED PACKAGES:***

   Developers can create their own packages to organize custom classes and interfaces according to their project structure and needs.

CREATING A PACKAGE IN JAVA

Now in order to create a package in java follow the certain steps as described below:

- First We Should Choose A Name For The Package We Are Going To Create And Include. The package command is The first line in the java program source code.
- Further inclusion of classes, interfaces, annotation types, etc that is required in the package can be made in the package. For example, the below single statement creates a package name called "FirstPackage".

Syntax: To declare the name of the package to be created. The package statement simply defines in which package the classes defined belong.

```
package FirstPackage ;
```

Implementation: To Create a Class Inside A Package

- First Declare The Package Name As The First Statement Of Our Program.
- Then We Can Include A Class As A Part Of The Package.

Example 1: Creating a Class Inside a Package

// Name of package to be created

```
package mypack; // package declaration

public class MyClass {
```

```
    public void display() {
        System.out.println("Hello from MyClass in mypack!");
    }
}
```

So Inorder to generate the above-desired output first do use the commands as specified use the following specified commands

Procedure to Generate Output:

1. Compile the MyClass.java file:

Command: javac MyClass.java

2. This command creates a MyClass.class file. To place the class file in the appropriate package directory, use:

Command: javac -d . MyClass.java

3. This command will create a new folder called mypack. To run the class, use:

Command: java mypack.MyClass

Output: The Above Will Give The Final Output Of The Example Program.

HOW TO IMPORT PACKAGES IN JAVA?

Java has an import statement that allows you to import an entire package (as in earlier examples), or use only certain classes and interfaces defined in the package.

The general form of import statement is:

```
import package.name.ClassName;   // To import a certain class only
import package.name.*   // To import the whole package
```

For example,

```
import java.util.Date; // imports only Date class
import java.io.*;      // imports everything inside java.io package
```

The import statement is optional in Java.

If you want to use class/interface from a certain package, you can also use its fully qualified name, which includes its full package hierarchy.

```
import mypack.MyClass; // import the class

public class Test {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.display();
    }
}
Output: Hello from MyClass in mypack!
```

CLASSPATH VARIABLE IN JAVA

The CLASSPATH variable in Java is an environment variable or a command-line option that specifies the locations where the Java Virtual Machine (JVM) and the Java compiler (javac) should search for user-defined classes and packages, as well as third-party libraries.

Here are the key aspects of the CLASSPATH variable:

- **Purpose:** It instructs the JVM and Java compiler where to find compiled Java bytecode files (ending with .class) and Java Archive (JAR) files, which contain collections of compiled classes.

- **Contents:** The CLASSPATH typically contains a list of directories and/or JAR file paths, separated by a platform-specific delimiter (e.g., semicolon ; on Windows, colon : on Unix-like systems).
- **Usage:**
  - **Environment Variable:** You can set CLASSPATH as a system-wide environment variable, making its settings available to all Java applications run on that system.
  - **Command-line Option:** You can also specify the CLASSPATH when running a Java program using the -classpath or -cp option with the java command, or when compiling with javac. This approach overrides any system-wide CLASSPATH settings for that specific execution.

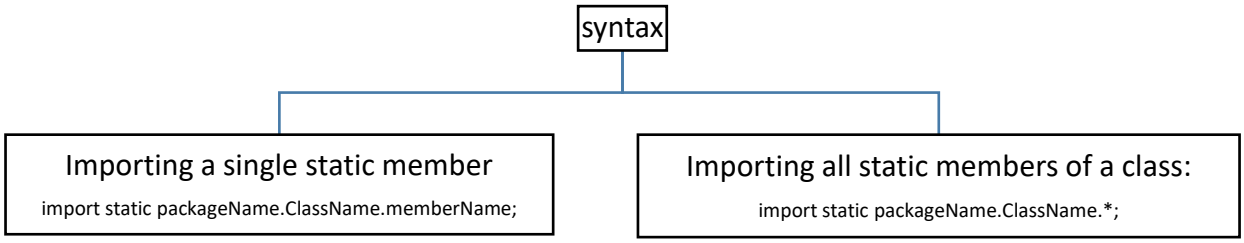
```
java -classpath "C:\myclasses;C:\lib\mylibrary.jar" MyMainClass
```

STATIC IMPORT IN JAVA

In Java, static import concept is introduced in 1.5 version. With the help of static import, we can access the static members of a class directly without class name or any object. For Example: we always use sqrt() method of Math class by using Math class i.e. Math.sqrt(), but by using static import we can access sqrt() method directly.

ADVANTAGE OF STATIC IMPORT: *LESS CODING IS REQUIRED.*

DISADVANTAGE OF STATIC IMPORT: *IT MAKES THE PROGRAM UNREADABLE AND UNMAINTAINABLE*




```
// Java Program to illustrate calling of predefined methods without static import
class Geeks {
   public static void main(String[] args)
   {
      System.out.println(Math.sqrt(4));
      System.out.println(Math.pow(2, 2));
      System.out.println(Math.abs(6.3));
   }
}
Output:
2.0
4.0
6.3
```

```
// Java Program to illustrate calling of predefined methods with static import
import static java.lang.Math.*;
class Test2 {
   public static void main(String[] args)
```

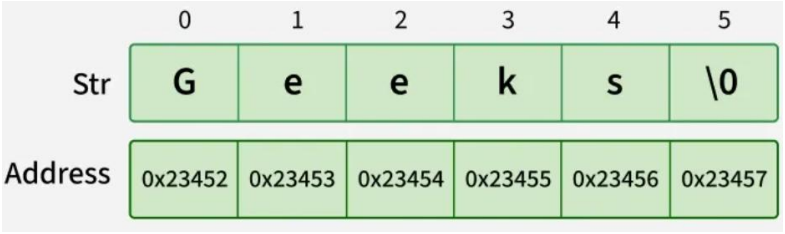
```
    {
        System.out.println(sqrt(4));
        System.out.println(pow(2, 2));
        System.out.println(abs(6.3));
    }
}
Output:
2.0
4.0
6.3
```

```
// Java to illustrate calling of static member of System class without Class name import static
java.lang.Math.*;
import static java.lang.System.*;
class Geeks {
    public static void main(String[] args)
    {
        // We are calling static member of System class directly without System class name
        out.println(sqrt(4));
        out.println(pow(2, 2));
        out.println(abs(6.3));
    }
}
Output:
2.0
4.0
6.3
```

NOTE: System is a class present in java.lang package and out is a static variable present in System class. By the help of static import we are calling it without class name.

STRINGS IN JAVA

In Java, a String is the type of object that can store a sequence of characters enclosed by double quotes and every character is stored in 16 bits, i.e., using UTF 16-bit encoding. A string acts the same as an array of characters.



We use double quotes to represent a string in Java. For example,

```
// create a string
String type = "Java programming";
```

Here, we have created a string variable named type. The variable is initialized with the string Java Programming.

CREATING A STRING IN JAVA

There are two ways to create a string in Java:

- String Literal
- Using new Keyword

1. STRING LITERAL (STATIC MEMORY)

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

Example:

```
String name = “Aadil”;
```

2. USING NEW KEYWORD (HEAP MEMORY)

```
String name = new String("Aadil");
```

In such a case, JVM will create a new string object in normal heap memory and the literal "Aadil" will be placed in the string constant pool. The variable name will refer to the object in the heap.

Example of strings:

```
class Main {
  public static void main(String[] args) {

    // create strings
    String first = "Java";
    String second = "Python";
    String third = "JavaScript";

    // print strings
    System.out.println(first);   // print Java
    System.out.println(second);  // print Python
    System.out.println(third);   // print JavaScript
```

```
  }
}
```

Note: Strings in Java are not primitive types (like int, char, etc). Instead, all strings are objects of a predefined class named String.

And all string variables are instances of the String class.

JAVA STRING OPERATIONS

Java provides various string methods to perform different operations on strings. We will look into some of the commonly used string operations.

1. GET THE LENGTH OF A STRING

To find the length of a string, we use the length() method. For example,

```
class Main {
  public static void main(String[] args) {

    String greet = "Hello! World";
    int length = greet.length();

    System.out.println("Length: " + length);
  }
}
Output: Length: 12
```

In the above example, the length() method calculates the total number of characters in the string and returns it.

2. JOIN TWO JAVA STRINGS

We can join two strings in Java using the concat() method. For example,

```
class Main {
  public static void main(String[] args) {

    String first = "Java ";
    String second = "Programming";

    // join two strings
    String joinedString = first.concat(second);

    System.out.println("Joined String: " + joinedString);
  }
}
Output: Joined String: Java Programming
```

We can also join two strings using the + operator in Java.

3. COMPARE TWO STRINGS

In Java, we can make comparisons between two strings using the equals() method. For example,

```
class Main {
  public static void main(String[] args) {

    // create 3 strings
    String first = "java programming";
    String second = "java programming";
    String third = "python programming";
```

```
    // compare first and second strings
    boolean result1 = first.equals(second);

    System.out.println("Strings first and second are equal: " + result1);

    // compare first and third strings
    boolean result2 = first.equals(third);

    System.out.println("Strings first and third are equal: " + result2);
  }
}
```

Output

Strings first and second are equal: true

Strings first and third are equal: false

The equals() method checks the content of strings while comparing them.

Note: We can also compare two strings using the == operator in Java. However, this approach is different than the equals() method.

Accessing Characters

```
String str = "Hello";
System.out.println(str.charAt(0)); // H
System.out.println(str.charAt(4)); // o
```

Substring

```
String str = "Programming";
System.out.println(str.substring(0, 6)); // Progra
System.out.println(str.substring(6));    // mming
```

Comparison

```
String s1 = "Java";
String s2 = "java";

System.out.println(s1.equals(s2));          // false
System.out.println(s1.equalsIgnoreCase(s2));// true
System.out.println(s1.compareTo(s2));       // negative value (lexicographic)
```

Searching

```
String str = "Java Programming";
System.out.println(str.indexOf("a"));     // 1
System.out.println(str.lastIndexOf("a")); // 13
System.out.println(str.contains("Pro"));  // true
```

Changing Case

```
String str = "Java";
System.out.println(str.toUpperCase()); // JAVA
System.out.println(str.toLowerCase()); // java
```

Trimming Spaces

```
String str = "   Hello World   ";
System.out.println(str.trim()); // "Hello World"
```

Replace Characters

```
String str = "Java";
System.out.println(str.replace('a', 'o')); // Jovo
```

ESCAPE CHARACTER IN JAVA STRINGS

The escape character is used to escape some of the characters present inside a string.

Suppose we need to include double quotes inside a string.

// include double quote

```
String example = "This is the "String" class";
```

Since strings are represented by double quotes, the compiler will treat "This is the " as the string. Hence, the above code will cause an error.

To solve this issue, we use the escape character \ in Java. For example,

// use the escape character

```
String example = "This is the \"String\" class.";
```

Now escape characters tell the compiler to escape double quotes and read the whole text.

IMMUTABLE AND MUTABLE STRINGS IN JAVA

In Java, the concept of mutable and immutable strings refers to whether the content of a string object can be altered after its creation.

IMMUTABLE STRINGS (STRING CLASS):

Definition: String objects in Java are immutable. This means that once a String object is created, its sequence of characters cannot be changed.

Behavior: Any operation that appears to modify a String (e.g., concatenation, replacement) actually creates a new String object with the modified content, leaving the original String object unchanged.

Example:

```
public class ImmutableDemo {
    public static void main(String[] args) {
        String str = "Java";

        // Trying to change the string
        str.concat(" Programming");

        // str remains unchanged
        System.out.println("Original String: " + str);

        // But if we assign the result to a new variable
        String newStr = str.concat(" Programming");
        System.out.println("Modified String: " + newStr);
    }
}
```

Advantages: Immutability offers advantages in terms of thread safety, security, and the ability to be used effectively in collections as keys (due to consistent hash codes).

MUTABLE STRINGS (STRINGBUFFER AND STRINGBUILDER CLASSES):

Definition: StringBuffer and StringBuilder objects are mutable. This means their content can be modified in place without creating new objects for each modification.

Behavior: Methods like append(), insert(), delete(), and replace() directly alter the character sequence within the existing object.

Example (StringBuilder):

```
public class MutableDemo {
   public static void main(String[] args) {
      // Using StringBuilder
      StringBuilder sb = new StringBuilder("Java");

      sb.append(" Programming"); // modifies the same object
      sb.insert(0, "Learn ");    // insert at beginning
      sb.replace(5, 9, "C++");   // replace part of the text
      sb.delete(0, 6);           // delete part of the text

      System.out.println("Final String: " + sb);
   }
}
```

DIFFERENCES BETWEEN STRINGBUFFER AND STRINGBUILDER:

StringBuffer is thread-safe (synchronized), making it suitable for multi-threaded environments but potentially slower.

StringBuilder is not thread-safe (not synchronized), making it generally faster than StringBuffer for single-threaded operations.

Advantages: Mutability is beneficial for scenarios requiring frequent string modifications, as it avoids the overhead of creating numerous new String objects.

STRINGS TRAVERSAL IN JAVA

```
public class StringTraversal1 {
   public static void main(String[] args) {
      String str = "Hello";

      for (int i = 0; i < str.length(); i++) {
         System.out.println(str.charAt(i));
      }
   }
}
```

STRING REVERSAL

```
public class StringReverseTraversal {
   public static void main(String[] args) {
      String str = "Reverse";
      for (int i = str.length() - 1; i >= 0; i--) {
         System.out.print(str.charAt(i));
      }
   }}
```

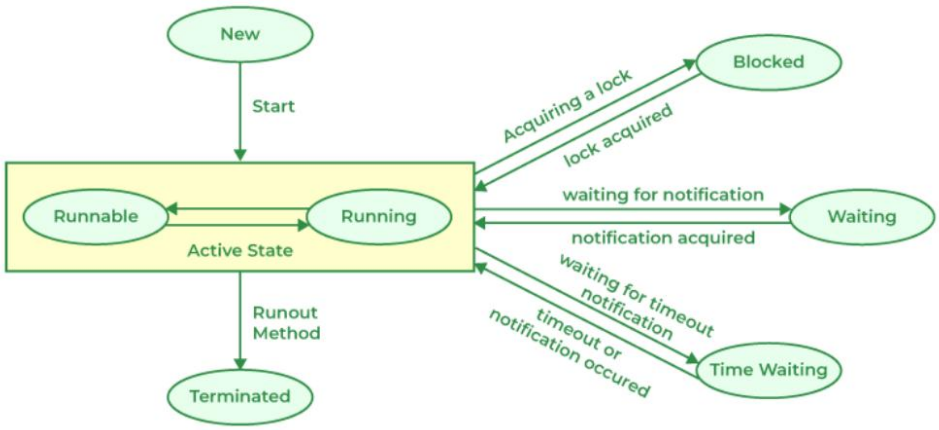
THREADS IN JAVA

A Java thread is the smallest unit of execution within a program. It is a lightweight sub-process that runs independently but shares the same memory space of the process, allowing multiple tasks to execute concurrently. For example: In MS Word, one thread formats the document while another takes user input.

LIFE CYCLE OF THREAD

During its lifetime, a thread transitions through several states, they are:

- New State
- Active State
- Waiting/Blocked State
- Timed Waiting State
- Terminated State



WORKING OF THREAD STATES

1. **New State:** By default, a thread will be in a new state, in this state, code has not yet started execution.
2. **Active State:** When a thread calls the **start()** method, it enters the Active state, which has two sub-states:
    a. Runnable State: The thread is ready to run but is waiting for the Thread Scheduler to give it CPU time.
    b. Running State: When the scheduler assigns CPU to a runnable thread, it moves to the Running state. After its time slice ends, it goes back to the Runnable state, waiting for the next chance to run.
3. **Waiting/Blocked State:** a thread is temporarily inactive, it may be in the Waiting or Blocked state:
    a. **Waiting:** If thread T1 needs to use a camera but thread T2 is already using it, T1 waits until T2 finishes.
    b. **Blocked:** If two threads try to use the same resource at the same time, one may be blocked until the other releases it.

    When multiple threads are waiting or blocked, the Thread Scheduler decides which one gets CPU based on priority.
4. **Timed Waiting State:** Sometimes threads may face starvation if one thread keeps using the CPU for a long time while others keep waiting.
    For example: If T1 is doing a long important task, T2 may wait indefinitely. To avoid this, Java provides the Timed Waiting state, where methods like **sleep()** allow a thread to pause only for a fixed time. After the time expires, the thread gets a chance to run.

5. **Terminated State:** A thread enters the Terminated state when its task is finished or it is explicitly stopped. In this state, the thread is dead and cannot be restarted.

CREATE THREADS IN JAVA

We can create threads in java using two ways

- Extending Thread Class
- Implementing a Runnable interface

1. BY EXTENDING THREAD CLASS

Create a class that extends Thread and override the run() method with the code you want the thread to execute. Then, make an object of your class and call the start() method. This will run the run() method in a new thread.

```
import java.io.*;
import java.util.*;

class MyThread extends Thread
{
   // initiated run method for Thread
   public void run(){
            String str = "Thread Started Running...";
      System.out.println(str);
   }
}
public class Geeks
{
            public static void main(String args[]){
            MyThread t1 = new MyThread();
            t1.start();
   }
}

Output: Thread Started Running...
```

2. USING RUNNABLE INTERFACE

Create a class that implements Runnable and override the run() method with the code for the thread. Then, make a Thread object, pass your Runnable object to it, and call the start() method.

Example:

```
import java.io.*;
import java.util.*;

class MyThread implements Runnable
{
            // Method to start Thread
   public void run(){
            String str = "Thread is Running Successfully";
      System.out.println(str);
   }
}
public class Geeks{
   public static void main(String[] args){
      MyThread g1 = new MyThread();

      // initializing Thread Object
```

```
        Thread t1 = new Thread(g1);

            // Running Thread
            t1.start();
    }
}

Output: Thread is Running Successfully
```

Note: Extend Thread when you don't need to extend any other class. Implement Runnable when your class already extends another class (preferred in most cases).

WHAT IS MULTITHREADING?

Multithreading is a process of executing multiple threads simultaneously. It is used to obtain the multitasking. It consumes less memory and gives the fast and efficient performance.

SYNCHRONIZATION IN THREADS

Thread Synchronization in Java is a mechanism that ensures that only one thread can access a shared resource (like a variable, method, or object) at a time.
It is mainly used to prevent data inconsistency when multiple threads try to modify the same resource simultaneously.

WHY SYNCHRONIZATION?

- Without synchronization, two or more threads may interfere with each other.
- Prevents race condition.
- Synchronization solves this by locking the resource until the current thread finishes using it.

TYPES OF SYNCHRONIZATION IN JAVA

- Synchronized Method – Entire method is synchronized.
- Synchronized Block – A block of code inside a method is synchronized.
- Static Synchronization – Synchronization applied on static methods.

Example 1: Synchronized Method

```
class Counter {
    private int count = 0;

    // Synchronized method
    public synchronized void increment() {
        count++;
    }
    public int getCount() {
        return count;
    }
}
public class SyncExample {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        // Two threads incrementing the same counter
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
```

```
            counter.increment();
        }
    });

    t1.start();
    t2.start();

    t1.join();
    t2.join();
    System.out.println("Final Count: " + counter.getCount());
  }
}
```

✅ Because of synchronization, the final count will always be 2000. ❌ Without synchronization, the result may be less than 2000 due to race conditions.

🔹 Example 2: Synchronized Block

```
public class Counter {
  private int count = 0;

  public void increment() {
    // Only this block of code is synchronized on the `this` object.
    synchronized (this) {
      count++;
    }
  }
}
```

INTER-THREAD COMMUNICATION (ITC)

ITC, also known as cooperation, is the process by which synchronized threads communicate with each other. It helps to avoid wasteful "busy waiting," where a thread continuously checks for a condition to become true.

The primary mechanism for inter-thread communication in Java involves the **wait(), notify(), and notifyAll()** methods, all defined in the java.lang.Object class.

Here's how these methods facilitate inter-thread communication:

- **wait():** When a thread calls wait() on an object, it releases the lock on that object and enters a waiting state. It remains in this state until another thread calls notify() or notifyAll() on the same object. This prevents "busy waiting".
- **notify():** When a thread calls notify() on an object, it wakes up a single thread that is currently waiting on that object's monitor. Which specific thread gets woken up is not guaranteed and depends on the JVM's scheduling.
- **notifyAll():** When a thread calls notifyAll() on an object, it wakes up all threads that are currently waiting on that object's monitor.

EXAMPLE SCENARIO (PRODUCER-CONSUMER):

A common example illustrating inter-thread communication is the Producer-Consumer problem. One thread (the producer) generates data and places it into a shared buffer, while another thread (the consumer) retrieves data from the buffer. wait() and notify() are used to:

- Consumer wait(): If the buffer is empty, the consumer thread calls wait() to pause its execution until the producer adds data.
- Producer notify(): After the producer adds data to the buffer, it calls notify() (or notifyAll()) to wake up a waiting consumer thread.

- Producer wait(): If the buffer is full, the producer thread calls wait() to pause its execution until the consumer removes data.
- Consumer notify(): After the consumer removes data from the buffer, it calls notify() (or notifyAll()) to wake up a waiting producer thread.

I/O STREAMS

In Java, an **I/O (Input/Output) Stream** represents a source of input or a destination for output, such as files, network connections, or memory. They are divided into two main categories: Input Streams and Output Streams.

- Input Streams are used to read data from a source, such as FileInputStream for reading files or BufferedInputStream for reading data more efficiently.
- Output Streams are used to write data to a destination, such as a FileOutputStream for writing to files or a BufferedOutputStream for efficient writing.

BYTE STREAMS

Byte streams handle the I/O of 8-bit bytes. They are the most basic type of stream and are used for reading and writing raw binary data, such as images, audio files, and any other non-textual data. The core classes for byte streams are InputStream and OutputStream.

- **InputStream:** An abstract class that represents an input stream of bytes. It has methods like read() to read a single byte or an array of bytes.
- **OutputStream:** An abstract class that represents an output stream of bytes. It has methods like write() to write a single byte or an array of bytes.

Common subclasses of **InputStream** and **OutputStream** include **FileInputStream** and **FileOutputStream** for file-based I/O.

CHARACTER STREAMS

Character streams are used for handling 16-bit Unicode characters. They are designed for I/O operations involving text data, and they automatically handle the conversion between bytes and characters based on a specified character encoding (e.g., UTF-8, UTF-16). The primary classes are **Reader** and **Writer**.

- Reader: An abstract class for character input streams. It has methods like read() to read a single character or an array of characters.
- Writer: An abstract class for character output streams. It has methods like write() to write a single character or an array of characters.

Subclasses like FileReader and FileWriter are used for reading from and writing to text files. Using character streams is highly recommended for text-based I/O as it avoids character encoding issues that can arise with byte streams.

BUFFERED STREAMS

Buffered streams improve I/O performance by wrapping other streams and adding a buffer. Instead of reading or writing one byte or character at a time from the underlying stream, they read or write data in larger blocks, or buffers, which significantly reduces the number of calls to the underlying system.

This is an example of the Decorator design pattern, where one stream "decorates" another to add new functionality.

- **BufferedInputStream and BufferedOutputStream:** These are byte stream wrappers. They read/write data in chunks to an internal buffer, making I/O operations faster, especially when dealing with large files.
- **BufferedReader and BufferedWriter:** These are character stream wrappers. They buffer characters to improve the efficiency of text I/O operations.

DATA STREAMS

Data streams are a specialized type of byte stream used for writing and reading primitive data types (like int, double, boolean) and strings in a machine-independent way. This is crucial for applications that need to transfer data between different systems or save and retrieve data in a specific format. The two main classes are DataInputStream and DataOutputStream.

- **DataInputStream:** Wraps an InputStream to provide methods like readInt(), readDouble(), and readUTF() that read primitive types from the stream.
- **DataOutputStream:** Wraps an OutputStream to provide methods like writeInt(), writeDouble(), and writeUTF() that write primitive types to the stream.

OBJECT STREAMS

Object streams provide a way to serialize and deserialize objects, allowing entire objects to be written to and read from a stream. Serialization is the process of converting an object into a sequence of bytes, and deserialization is the reverse process. This is extremely useful for saving the state of an application, sending objects over a network, or caching them.

- **ObjectInputStream:** Reads objects from an underlying InputStream. The readObject() method is used to deserialize an object.
- **ObjectOutputStream**: Writes objects to an underlying OutputStream. The writeObject() method is used to serialize an object.

For an object to be serializable, its class must implement the java.io.Serializable marker interface. This interface has no methods, but it signals to the Java Virtual Machine (JVM) that the objects of this class can be converted into a stream of bytes.

STANDARD STREAMS IN JAVA

Standard streams in Java are predefined, system-level I/O streams automatically available to every Java program. They provide a simple and consistent way to handle input from and output to a console or command-line environment. These streams are part of the java.lang.System class and are represented by public static fields.

The three standard streams are:

1. STANDARD INPUT (SYSTEM.IN)

System.in is a byte stream of type InputStream. It represents the standard input device, which is typically the keyboard. You can use it to read data typed by a user. While it's a byte stream, it's often wrapped in a character stream or buffered stream to make reading text easier and more efficient, for example, using a Scanner or BufferedReader.

```
import java.util.Scanner;
public class StandardInputExample {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter your name: ");
        String name = sc.nextLine(); // takes input from keyboard
        System.out.println("Hello, " + name);
    }
}
```

2. STANDARD OUTPUT (SYSTEM.OUT)

System.out is a byte stream of type PrintStream. It represents the standard output device, which is usually the console or terminal. It's the most commonly used stream for printing output. The PrintStream class provides convenient methods like print(), println(), and printf() to output various data types.

```
public class StandardOutputExample {
    public static void main(String[] args) {
        System.out.println("This is standard output.");
        System.out.print("Hello from Java!");
    }
}
```

3. STANDARD ERROR (SYSTEM.ERR)

System.err is also a byte stream of type PrintStream. It's used to output error messages and diagnostic information. Like System.out, it typically prints to the console. The key difference is its purpose: it's used for error messages, which allows developers and users to separate normal program output from error messages.

```
public class StandardErrorExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // will cause an error
        } catch (ArithmeticException e) {
            System.err.println("Error: Division by zero is not allowed!");
        }
    }
}
```

FILE I/O IN JAVA – READING AND WRITING FILES EXPLAINED

In Java, **File I/O (Input and Output)** allows a program to **interact with files on the storage system** — reading data from files (input) and writing data to files (output). Understanding File I/O is essential for handling text files, configuration data, logs, and more.

1. FILE I/O BASICS

At the core of Java File I/O are **streams** — think of a stream as a **pathway of data** that connects your program to an external source (like a file, keyboard, or network).

Streams are of two main types:

Stream Type	Description	Use Case	Common Classes
Byte Streams	Handle data in **8-bit bytes**.	For binary data like images, videos, and audio files.	`InputStream`, `OutputStream`, `FileInputStream`, `FileOutputStream`
Character Streams	Handle data in **16-bit Unicode characters**.	For reading and writing **text files**.	`Reader`, `Writer`, `FileReader`, `FileWriter`, `BufferedReader`, `BufferedWriter`

MODERN FILE I/O (NIO.2)

Java 7 introduced the NIO.2 API (java.nio.file package), which made file handling simpler and more powerful.
Two key classes are:

- **`Path`** – Represents a file or directory path.
- **`Files`** – Provides static methods for common file operations such as reading, writing, copying, and deleting.

2. READING FROM A FILE (USING NIO.2)

Java's NIO.2 API allows you to read a file's content easily with the `Files` class.

Example: Reading an Entire File as Text

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;

public class FileReaderExample {
    public static void main(String[] args) {
        Path filePath = Path.of("sample.txt");

        try {
            // Reads the entire file content as a single String
            String content = Files.readString(filePath);
            System.out.println("File Content:\n" + content);
        } catch (IOException e) {
            System.err.println("Error reading file: " + e.getMessage());
        }
    }
}
```

Explanation:

- Path.of("sample.txt"): Defines the path to the file.
- Files.readString(): Reads all content using UTF-8 encoding and returns it as a string.
- A try-catch block handles possible IOException if the file doesn't exist or can't be read.

Note: For large files, use Files.lines(filePath) to read data line by line instead of loading it all at once.

3. WRITING TO A FILE (USING NIO.2)

You can also write data to files easily using the `Files.writeString()` method.

Example: Writing a String to a File

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.io.IOException;

public class FileWriterExample {
    public static void main(String[] args) {
        Path filePath = Path.of("output.txt");
        String content = "Hello Java File I/O!\nThis is a new line.";

        try {
            // Creates the file if it doesn't exist, and writes content to it
            Files.writeString(filePath, content, StandardOpenOption.CREATE);
            System.out.println("Data successfully written to output.txt");
        } catch (IOException e) {
            System.err.println("Error writing to file: " + e.getMessage());
        }
    }
}
```

Explanation:

- StandardOpenOption.CREATE: Creates the file if it doesn't exist.
- Use StandardOpenOption.APPEND to add data to the end instead of overwriting.

4. LEGACY APPROACH: USING CHARACTER STREAMS

Before NIO.2, Java used the `java.io` package for file handling with classes like `FileReader`, `FileWriter`, and their buffered versions.

Example: Reading Text Line by Line

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class BufferedReaderExample {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("sample.txt"))) {
            String line;
            System.out.println("Reading file line by line:");
            while ((line = reader.readLine()) != null) {
                System.out.println("-> " + line);
            }
        } catch (IOException e) {
            System.err.println("File I/O Error: " + e.getMessage());
        }
    }
}
```

Explanation:

- BufferedReader reads data efficiently by storing it temporarily in memory.
- try-with-resources automatically closes the stream, preventing memory leaks.

Example: Writing to a File Using BufferedWriter

```
import java.io.BufferedWriter;
```

```
import java.io.FileWriter;
import java.io.IOException;

public class BufferedWriterExample {
   public static void main(String[] args) {
      try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt", true))) {
         writer.write("\nAppended text using BufferedWriter.");
         writer.newLine();
         writer.write("End of file.");
      } catch (IOException e) {
         System.err.println("File I/O Error: " + e.getMessage());
      }
   }
}
```

EXPLANATION:

- The second argument true in FileWriter("output.txt", true) enables append mode.
- writer.newLine() adds a line break appropriate for the system.

5. COMMON EXCEPTIONS IN FILE I/O

Exception	Cause
FileNotFoundException	File does not exist or the path is invalid.
IOException	General input/output error during reading or writing.
SecurityException	Attempt to access a file without the required permissions.

Always handle these exceptions properly using try-catch or declare throws IOException.

WHAT IS SERIALIZATION IN JAVA?

Serialization in Java is the process of converting an object state into a byte stream. You can then store this byte stream in a file or transmit it over a network. This makes objects persistent.

- Serialization helps you save object data. Imagine you have an object holding user settings. You can serialize this object to a file. When the user opens your application again, you deserialize the object. This restores the user settings.
- Serialization also helps in distributed systems.

HOW SERIALIZATION WORKS

The java.io.Serializable interface is the core of serialization. Your class must implement this marker interface to be serializable. A marker interface has no methods to implement. It simply tells the JVM that objects of this class can be serialized.

You use the ObjectOutputStream class to serialize objects. This class writes Java objects to an output stream.

HERE IS HOW YOU SERIALIZE AN OBJECT:

- Create a class that implements Serializable. This tells Java the object can be converted to bytes.
- Create an ObjectOutputStream. This stream handles writing objects.
- Call writeObject() on the ObjectOutputStream. Pass the object you want to serialize.

Step-by-Step Example: Serializing an Object

Let us say you have a Student class. You want to save student data.

1. Create the Student Class:

```
import java.io.Serializable;
public class Student implements Serializable {
   private String name;
   private int age;
   private transient String secretPassword; // This field will not be serialized

   public Student(String name, int age, String secretPassword) {
      this.name = name;
      this.age = age;
      this.secretPassword = secretPassword;
   }
   public String getName() {
      return name;
   }
   public int getAge() {
      return age;
   }
   // You will not have a getter for secretPassword to emphasize transient
}
```

Notice the transient keyword for secretPassword. This tells the JVM to ignore this field during serialization.

2. Serialize the Student Object:

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;
public class SerializeStudent {
   public static void main(String[] args) {
      Student student = new Student("Alice", 20, "mySecurePass");

      try (FileOutputStream fileOut = new FileOutputStream("student.ser");
           ObjectOutputStream out = new ObjectOutputStream(fileOut)) {

         out.writeObject(student);
         System.out.println("Student object serialized successfully.");

      } catch (IOException i) {
         i.printStackTrace();
      }
   }
}
```

This code creates a Student object. It then writes the object to a file named student.ser. The try-with-resources statement ensures streams close automatically.

WHAT IS DESERIALIZATION IN JAVA?

Deserialization is the reverse process of serialization. It converts a byte stream back into an object. This reconstructs the object in memory.

HOW DESERIALIZATION WORKS

You use the ObjectInputStream class to deserialize objects. This class reads Java objects from an input stream.

Here is how you deserialize an object:

- Create an ObjectInputStream. This stream handles reading objects.
- Call readObject() on the ObjectInputStream. This method reads the serialized object.
- Cast the result to the original object type.

Step-by-Step Example: Deserializing an Object

Now, let us read the Student object back from the student.ser file.

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.IOException;
import java.lang.ClassNotFoundException;
public class DeserializeStudent {
   public static void main(String[] args) {
      Student student = null;

      try (FileInputStream fileIn = new FileInputStream("student.ser");
           ObjectInputStream in = new ObjectInputStream(fileIn)) {

         student = (Student) in.readObject();
         System.out.println("Student object deserialized successfully.");

         System.out.println("Name: " + student.getName());
         System.out.println("Age: " + student.getAge());
         // System.out.println("Secret Password: " + student.getSecretPassword()); // This would be null

      } catch (IOException i) {
         i.printStackTrace();
         return;
      } catch (ClassNotFoundException c) {
         System.out.println("Student class not found.");
         c.printStackTrace();
         return;
      }
   }
}
```

This code reads the student.ser file. It then casts the read object back to a Student type. You can then access its data. Notice that the secretPassword field is not available because it was marked transient.

BEST PRACTICES FOR SERIALIZATION

USE SERIALVERSIONUID

Implement serialVersionUID in your Serializable classes. This static final long field identifies the version of your serialized class. When you deserialize an object, the JVM compares the serialVersionUID of the class with the one in the serialized data. If they differ, it throws an InvalidClassException. This prevents compatibility issues when you change your class.

You can generate a serialVersionUID using your IDE or a command-line tool.

USE TRANSIENT KEYWORD

Use the transient keyword for fields you do not want to serialize. This includes sensitive data (like passwords) or data that can be recomputed (like a calculated average). This saves space and improves security.

THE DELEGATION EVENT MODEL

The delegation event model provides standard mechanisms to generate and process events.

A source generates an event and sends it to registered listeners.

Listeners wait for events, process them, and then return.

This design separates application logic from user interface logic, allowing UI elements to delegate event processing to separate code. Listeners must register with sources to receive notifications, ensuring events are only sent to interested listeners.

EVENTS

In this model, an event is an object describing a state change in a source. Events can be generated by user interactions (button presses, keyboard input, list selections, mouse clicks) or by non-user interface causes (timer expiration, counter thresholds, hardware/software failures). Developers can define application-specific events.

SOURCES OF EVENTS

A source is an object that generates events when it's internal state changes. Sources may generate multiple event types and must register listeners for each type using registration methods:

$$public\ void\ addTypeListener(TypeListener\ el)$$

Where, Type is the event name and el is the event listener reference (e.g., addKeyListener()).

When an event occurs, all registered listeners receive a copy of the event object, known as multicasting the event

The **Java 1.1 Event Delegation Model** (EDM) relies on three core object types to handle user interactions in a Graphical User Interface (GUI): the **Source**, the **Event**, and the **Listener**.[1]

1. EVENT SOURCE OBJECT

The Event Source is the GUI component (like a button, text field, or window) that generates an event when a user interacts with it. It acts as the dispatcher, notifying any interested parties that something has happened.

The Source object's primary role is to **maintain a list of registered listeners** and to **notify** them when the event occurs.

Associated Methods

Source objects provide methods for managing the list of interested listeners. These methods follow a standard naming convention:

Method Type	Purpose	Naming Convention	Example (for a button)
Registration	Adds a Listener to the Source's internal list.	add*Type*Listener(Listener l)	myButton.addActionListener(myHandler);

Method Type	Purpose	Naming Convention	Example (for a button)
Deregistration	Removes a Listener from the Source's internal list.	remove*Type*Listener(Listener l)	myButton.removeActionListener(myHandler);

Example: If you have a JButton named saveButton, it is an event source for an ActionEvent.

2. EVENT OBJECT

The Event Object is a class instance that encapsulates all the specific information about the event that just occurred. It is created by the Event Source when the interaction happens.

The Event object serves as the messenger between the Source and the Listener. It tells the Listener what happened, where it happened, and any associated data (like mouse coordinates or the key pressed).

All event objects in AWT/Swing inherit from the base class java.util.EventObject.4

Associated Methods

Event objects provide methods to retrieve details about the event. Since all events inherit from the base class, they share a few core methods, plus specific methods for their event type.

Method	Purpose	Example
`Object getSource()`	Returns the **object** (the GUI component) that generated the event.	`Object source = e.getSource();`
`String toString()`	Returns a string representation of the event.	`System.out.println(e.toString());`
`int getClickCount()`	(Specific to `MouseEvent`) Returns the number of consecutive mouse clicks.	`int clicks = e.getClickCount();`
`int getKeyCode()`	(Specific to `KeyEvent`) Returns the virtual key code of the key pressed.	`int key = e.getKeyCode();`

Example: When a user clicks a button, the source creates an `ActionEvent` object.[5] This object contains a reference to the button itself (`getSource()`).

3. EVENT LISTENER OBJECT

The **Event Listener** (or Event Handler) is a specific object that is **interested in and capable of processing** a particular type of event.

The Listener's primary role is to **execute the custom application logic** (the code) in response to the event. It implements a specific **Listener Interface** that dictates the method(s) it must contain.

Associated Methods

Listener objects must implement the method(s) defined in their corresponding interface. These are the methods that the **Event Source calls** to notify the Listener.

Listener Interface	Key Method (The Handler)	When is it called?
`ActionListener`	`void actionPerformed(ActionEvent e)`	When a button is clicked or Enter is pressed in a text field.
`MouseListener`	`void mouseClicked(MouseEvent e)`	When a mouse button is pressed and released without moving.
`KeyListener`	`void keyPressed(KeyEvent e)`	When a key is pressed down.

Example: To handle a button click, your listener class must implement the `ActionListener` interface and provide the code inside the `actionPerformed(ActionEvent e)` method

1. LOW-LEVEL EVENTS (RAW INPUT)

Low-Level Events represent direct, raw physical input from the user's operating system (OS) devices, such as the mouse, keyboard, or window system. They are the most granular events and capture every single step of an interaction.

Characteristics:

- **Abstraction Level:** Low-Level. They describe how the user interacted physically.
- **Focus:** Physical actions, device state changes, and component lifecycle events.4
- **Frequency:** High. A single high-level action (like a button click) often generates multiple low-level events (mouse down, mouse up, mouse click).5
- **Use Case:** Needed when you require precise control over every detail of the interaction, such as custom drawing, drag-and-drop, or tracking exact mouse coordinates.6

Examples

Event Class	Listener Interface	Description
`MouseEvent`	`MouseListener`, `MouseMotionListener`	Pressing a mouse button, releasing a button, moving the mouse, dragging the mouse.
`KeyEvent`	`KeyListener`	Pressing a key down, releasing a key, or typing a character.

Event Class	Listener Interface	Description
WindowEvent	WindowListener	A window opening, closing, minimizing (iconifying), or maximizing.
FocusEvent	FocusListener	A component gaining or losing input focus.
ComponentEvent	ComponentListener	A component being moved, resized, or made visible/invisible.

2. SEMANTIC EVENTS (MEANINGFUL ACTIONS)

Semantic Events (also called High-Level Events) represent a meaningful, completed action performed by the user on a specific GUI component. They abstract away the physical details of the input and describe the intent or the semantics of the user's interaction.

Characteristics:

- **Abstraction Level:** High-Level. They describe what the user intended to do.
- **Focus**: The logical consequence or state change of a component's model.8
- **Source**: They are often translated by the component from a sequence of low-level events. For example, a button listens for a mouse down and a mouse up inside its area, then fires one ActionEvent.
- **Use Case**: Preferred for general application logic, as they are simpler to handle and are look-and-feel (L&F) independent. They ensure the component responds correctly whether activated by a mouse click, a keyboard shortcut (e.g., the Spacebar), or another L&F-specific gesture.

Examples

Event Class	Listener Interface	Meaningful Action
ActionEvent	ActionListener	A button was clicked, a menu item was selected, or **Enter** was pressed in a text field.
ItemEvent	ItemListener	The state of an item changed (e.g., selecting/deselecting a checkbox or radio button).
AdjustmentEvent	AdjustmentListener	The value of a scrollbar was adjusted (manually or programmatically).
TextEvent	TextListener	The text content of a text area or text field changed.